DELTA – Střední škola informatiky a ekonomie, s.r.o.
Ke Kamenci 151, Pardubice

LifeMeter

Mobilní aplikace pro sledování zdravotních metrik

Autor: Zadrobílek Jan
Třída: 4.B
Studijní obor: Informační technologie 18-20-M/01
Školní rok: 2025/2026

Zadání maturitního projektu z informatických předmětů

Jméno a příjmení:	*Jan Zadrobílek*
Školní rok:	*2025/2026*
Třída:	*4.B*
Obor:	*Informační technologie 18-20-M/01*
Téma práce:	*Mobilní aplikace „LifeMeter“*
Vedoucí práce:	*Ing. Monika Borkovcová, Ph.D.*

Způsob zpracování, cíle práce, pokyny k obsahu a rozsahu práce:

Cílem projektu je vytvoření mobilní aplikace pro OS Android, která bude sloužit jako nástroj pro sledování fyzické aktivity, nutričních hodnot a spánku. Funkce budou zahrnovat možnost přihlášení, notifikace a lokální manipulaci s daty. Aplikace bude navržena tak, aby poskytovala uživatelům intuitivní uživatelské rozhraní a efektivní způsob interakce. Aplikace je funkční pro OS Android 10 a vyšší.

Popis funkčnosti mobilní aplikace, která umožňuje:

Záznam fyzických aktivit

- Možnost přidávat aktivity s informacemi jako poloha nebo poznámka.

Sledování výživy

- Přidávání zkonzumovaného jídla, doplňků stravy, sledování kalorií, makroživin a vitamínů.

Sledování spánku

- Záznam délky spánku s možností přidání poznámek, případně možnost pracovat s kvalitou spánku.

Správa uživatelů

- Přihlašování bude probíhat pomocí emailu se zabezpečeným ověřením.

Offline

- Aplikace bude umožňovat offline režim se synchronizací dat při obnovení připojení.

Notifikace

- Upozornění ke cvičení a spánku.

Integrace s jinými aplikacemi

- Aplikace bude umožňovat propojení s vybranou aplikací/platformou.

Prohlašuji, že jsem maturitní projekt vypracoval(a) samostatně, výhradně s použitím uvedené literatury.

V Pardubicích 26. 3. 2026 .....

(vlastnoruční podpis)

Rád bych upřímně poděkoval své vedoucí práce, Ing. Monice Borkovcové, za odborné vedení, rady, trpělivost a čas, který mi věnovala při konzultacích a zpracování tohoto maturitního projektu.

Resumé

Tato práce představuje aplikaci „LifeMeter“, určenou pro sledování a analýzu životních metrik uživatele za účelem osobního rozvoje a zlepšení zdraví. Aplikace umožňuje uživatelům centralizované zaznamenávání a monitorování stravy, kvality spánku a fyzické aktivity. Součástí výsledného systému je vedle mobilního klienta také doprovodná webová vrstva sloužící jako veřejná prezentace, centrum verzovaného stahování mobilních buildů a administrátorská konzole pouze pro čtení.

Backendová část systému je postavena na technologiích Bun, Hono a relační databázi PostgreSQL. Pro komunikaci mezi klientskými aplikacemi a serverem je implementováno REST API, které zajišťuje autentizaci, synchronizaci zdravotních dat, správu uživatelských profilů a administrativní přehled nad systémem.

Mobilní klientská aplikace je vyvinuta pomocí technologií React Native a Expo. Zvolený přístup zaručuje kompatibilitu a zároveň umožňuje přístup k nativním rozhraním pro synchronizaci zdravotních dat, konkrétně HealthKit pro operační systém iOS a Health Connect pro platformu Android.

Webová část je implementována pomocí frameworku Next.js a knihovny React. Veřejné stránky poskytují informace o produktu a verzované stažení aktuálních i archivních buildů, zatímco administrátorské routy zpřístupňují přehled uživatelů a logů nad produkčním prostředím.

Klíčová slova: *mobilní aplikace, zdravotní metriky, React Native, Expo, Next.js, REST API, Bun, Hono, PostgreSQL*

Summary

This thesis presents the „LifeMeter“ application, designed for tracking and analyzing a user’s lifestyle metrics to support personal development and health improvement. The application allows users to centrally log and monitor their diet, sleep quality, and physical activity. In addition to the mobile client, the resulting system also includes a companion web layer that serves as a public presentation site, a versioned mobile download center, and a read-only administration console.

The backend of the system is built on Bun, Hono, and a PostgreSQL relational database. A REST API is implemented for communication between the clients and the server, providing authentication, health data synchronization, user profile management, and operational administration features.

The mobile client application is developed using React Native and Expo. The chosen approach ensures cross-platform compatibility while enabling access to native APIs for

health data synchronization, specifically HealthKit for the iOS operating system and Health Connect for Android.

The web layer is implemented with Next.js and React. Public routes present the product and expose versioned current and archived builds, while the administration routes provide a production overview of users and logs.

Keywords: *mobile application, health metrics, React Native, Expo, Next.js, REST API, Bun, Hono, PostgreSQL*

Obsah

Úvod **8**

1 Teoretická část **9**

1.1 Současný stav . . . 9

1.2 Existující řešení . . . 10

1.2.1 Google Fit . . . 10

1.2.2 Apple Health . . . 10

1.2.3 MyFitnessPal . . . 10

1.2.4 Cronometer . . . 10

1.2.5 Sleep Cycle . . . 11

1.3 Volba nástrojů a zdůvodnění . . . 11

1.3.1 React Native a Expo . . . 11

1.3.2 Next.js pro webovou vrstvu . . . 11

1.3.3 Bun a Hono . . . 12

1.3.4 PostgreSQL . . . 12

1.3.5 Better Auth . . . 12

1.3.6 Docker a Oracle Cloud . . . 12

2 Metodika a vlastní řešení **14**

2.1 Architektura systému . . . 14

2.1.1 Klientská vrstva . . . 14

2.1.2 Webová vrstva . . . 15

2.1.3 Serverová vrstva . . . 15

2.1.4 Datová vrstva . . . 16

2.2 Datový model . . . 16

2.2.1 Autentizace . . . 16

2.2.2 Výživa . . . 16

2.2.3 Spánek . . . 16

2.2.4 Tréninky . . . 18

2.2.5 Uživatelský profil . . . 18

2.2.6 Synchronizace zdravotních dat . . . 18
2.3 Popis řešení . . . 19
2.3.1 Mobilní aplikace . . . 19
2.3.2 Serverová část . . . 20
2.3.3 CI/CD pipeline . . . 21
2.4 Návrh aplikace . . . 22
2.4.1 Případy užití . . . 22
2.4.2 Diagram navigace . . . 22
2.4.3 Diagram tříd . . . 23
2.5 Popis důležitých algoritmů . . . 25
2.5.1 Výpočet metabolismu a denního energetického výdeje . . . 25
2.5.2 Stanovení cílového denního kalorického příjmu . . . 26
2.5.3 Výpočet nutričních hodnot jídla . . . 27
2.5.4 Fulltextové vyhledávání potravin . . . 27
2.5.5 Offline fronta požadavků . . . 28

Výsledky **30**
2.6 Splnění požadavků zadání . . . 30
2.7 Implementované moduly . . . 31
2.8 Technické parametry . . . 33
2.9 Uživatelské rozhraní . . . 33
2.10 Testování . . . 36

Diskuze **37**
2.11 Zhodnocení dosažených výsledků . . . 37
2.12 Zdůvodnění volby technologií . . . 37
2.12.1 React Native a Expo . . . 37
2.12.2 Bun a Hono . . . 37
2.12.3 PostgreSQL a volba relační databáze . . . 38
2.12.4 Better Auth a správa relací . . . 38
2.12.5 MMKV pro offline frontu . . . 38
2.13 Srovnání s existujícími řešeními . . . 38
2.14 Omezení implementace . . . 39
2.15 Možná budoucí rozšíření . . . 39

Závěr **41**

Literatura **42**

Seznam obrázků **43**

Seznam tabulek **44**

Seznam příloh **47**

A: Diagramy
B: Tabulky
C: Schémata
D: API dokumentace
E: Instalační návod
2.16 Tabulky 48

Úvod

Sledování zdravotních metrik se v posledních letech stalo běžnou součástí každodenního života stále většího počtu lidí. Rozšíření chytrých telefonů a nositelných zařízení umožnilo sbírat data o fyzické aktivitě, výživě či spánku způsobem, který byl dříve dostupný pouze ve specializovaných zařízeních. Zájem o oblast takzvaného *quantified self* – tedy systematického zaznamenávání vlastních biologických a behaviorálních dat za účelem osobního rozvoje – neustále roste [1].

Stávající řešení na trhu (např. MyFitnessPal, Google Fit či Hevy) se často zaměřují jen na dílčí aspekty zdraví a uživatel je nucen přecházet mezi více nepropojenými aplikacemi, mnohdy s omezenými funkcemi v neplacených verzích. Cílem tohoto maturitního projektu je proto návrh a implementace aplikace *LifeMeter*, která usiluje o centralizaci sledovaných metrik (fyzické aktivity, výživa, spánek, hmotnost) do jediného prostředí s přehlednou domovskou obrazovkou.

Mobilní klient pro systémy Android a iOS je vyvinut pomocí frameworků React Native [2] a Expo [3]. Synchronizaci dat ze systémových úložišť lokálních zařízení zajišťují rozhraní Health Connect a HealthKit. Serverová část, komunikující s klientem přes REST API, je postavena na moderním běhovém prostředí Bun [4] s využitím frameworku Hono a relační databáze PostgreSQL. Součástí výsledného systému je také webová vrstva postavená na frameworku Next.js, která slouží jako veřejná prezentace, centrum stahování verzovaných mobilních buildů a administrátorská konzole pouze pro čtení. Serverový i webový stack je kontejnerizován v prostředí Docker a nasazen na platformě Oracle Cloud.

1. Teoretická část

1.1 Současný stav

Moderní mobilní operační systémy poskytují vývojářům třetích stran standardizovaná rozhraní pro přístup ke zdravotním datům uloženým v centralizovaném systémovém úložišti. Na platformě Android tuto roli od verze Android 8 zastává Health Connect [5], na platformě iOS pak HealthKit [6], dostupný od verze iOS 8. Obě rozhraní fungují na principu zprostředkování – aplikace nezíská přímý přístup k datům jiných aplikací, nýbrž čte z a zapisuje do sdíleného úložiště, přičemž uživatel musí explicitně udělit oprávnění ke každému datovému typu zvlášť. Tento model výrazně zjednodušuje integraci mezi aplikacemi (sledování kroků, chytré hodinky, fitness tracker) a eliminuje nutnost vytvářet proprietární synchronizační protokoly.

Klíčovým modelem při návrhu mobilních aplikací závislých na vzdáleném serveru je přístup *offline-first* [7]. Mobilní zařízení jsou ze své podstaty vystavena nestabilnímu nebo zcela chybějícímu síťovému připojení, proto robustní aplikace nepředpokládá nepřetržitou konektivitu, nýbrž primárně pracuje s lokálně uloženými daty a se serverem se synchronizuje, jakmile je připojení k dispozici. Na platformě React Native je standardním řešením pro rychlé lokální úložiště knihovna MMKV [8], která využívá nativní implementaci ve formě sdílené paměti mapované na soubor a dosahuje výrazně nižší latence než `AsyncStorage`.

Komunikace mezi mobilním klientem a serverovou částí je v současných aplikacích zpravidla realizována prostřednictvím architektury REST (*Representational State Transfer*) [9]. REST definuje sadu architektonických omezení – bezstavovost, jednotné rozhraní, vrstvený systém – která vedou k interoperabilním, škálovatelným a snadno testovatelným API. Každý zdroj je identifikován URL adresou, operace nad ním jsou vyjádřeny standardními HTTP metodami (GET, POST, PUT, DELETE) a data jsou přenášena zpravidla ve formátu JSON.

Autentizace uživatelů v moderních webových a mobilních aplikacích se nejčastěji řeší dvěma přístupy. Prvním jsou tokeny JWT (*JSON Web Token*), které jsou bezstavové a snadno horizontálně škálovatelné, avšak jejich uložení na straně klienta přináší bezpečnostní rizika, pokud jsou uchovávány v přístupném úložišti. Druhým přístupem jsou stavové relace spravované serverem s identifikátorem uloženým v `HttpOnly` cookie, která

není přístupná JavaScriptovým kódem, čímž je eliminováno riziko krádeže tokenu skriptem třetí strany [10].

1.2 Existující řešení

Na trhu existuje řada aplikací zaměřených na sledování zdravotních metrik. Níže jsou popsána nejrozšířenější řešení včetně jejich omezení, jež vedla k rozhodnutí vytvořit vlastní aplikaci.

1.2.1 Google Fit

Google Fit [11] je platforma vyvinutá společností Google, integrovaná přímo do operačního systému Android. Automaticky zaznamenává pohybovou aktivitu prostřednictvím senzorů zařízení a slouží jako centrální úložiště zdravotních dat pro ekosystém Android. Aplikace nabízí jednoduché přehledy a cíle, avšak detailní sledování výživy ani spánku nativně nepodporuje – tyto oblasti vyžadují napojení na specializované aplikace třetích stran.

1.2.2 Apple Health

Apple Health [12] zastává v ekosystému iOS analogickou roli jako Google Fit na Androidu – jde o systémové úložiště zdravotních dat přístupné přes rozhraní HealthKit. Aplikace agreguje záznamy z různých zdrojů a zobrazuje je v přehledném rozhraní. Platforma je dostupná výhradně na zařízeních Apple a funguje primárně jako datový hub; vlastní zadávání výživy nebo tréninkových plánů není nativně podporováno.

1.2.3 MyFitnessPal

MyFitnessPal [13] je jednou z nejrozšířenějších aplikací pro sledování výživy na světě. Disponuje rozsáhlou databází potravin, podporou skenování čárových kódů a integrací s fitness zařízeními. Sledování tréninkového zatížení je omezeno na kalorický výdej bez možnosti podrobné analýzy silových tréninků. Velká část funkcionalit – pokročilé makroanalýzy, plánování, hloubkové statistiky – je dostupná pouze v placené verzi Premium.

1.2.4 Cronometer

Cronometer [14] se od konkurence odlišuje důrazem na nutriční přesnost. Databáze potravin vychází primárně z ověřených vědeckých zdrojů (USDA, NCCDB) a aplikace sleduje vedle makroživin také mikroživiny, vitamíny a minerály. Sledování fyzické aktivity a spánku je v základní verzi omezeno; pokročilé funkce jsou vázány na placenou verzi Gold.

1.2.5 Sleep Cycle

Sleep Cycle [15] je aplikace specializovaná výhradně na analýzu kvality spánku. Ke sledování využívá mikrofon zařízení nebo propojení s chytrými hodinkami a na základě detekce pohybu a zvukových vzorů odhaduje spánkové fáze. Nabízí chytrý budík probouzející uživatele v nejlehčí fázi spánku. Aplikace nenabízí žádnou integraci se sledováním výživy či pohybu a pokročilé statistiky jsou dostupné pouze v rámci předplatného.

1.3 Volba nástrojů a zdůvodnění

Výběr technologií byl proveden s ohledem na požadavky projektu, a osobní znalostí s některými z technologií.

1.3.1 React Native a Expo

Pro vývoj mobilního částí byl zvolen framework React Native [2] v kombinaci s platformou Expo [3]. React Native umožňuje sestavit jedinou sdílenou kódovou základnu v jazyce JavaScript/TypeScript, ze které jsou generovány nativní aplikace pro Android i iOS. Tato vlastnost byla klíčovým kritériem výběru – projekt byl sice primárně navrhován pro Android, avšak podpora iOS je zachována pro případnou budoucí expanzi bez nutnosti přepisovat aplikaci od základu. Rozhodujícím faktorem byla také předchozí znalost ekosystému React, jež výrazně zkrátila dobu potřebnou k osvojení frameworku.

Jako alternativy připadaly v úvahu framework Flutter a nativní vývoj v jazycích Kotlin resp. Swift. Flutter sice nabízí srovnatelnou multiplatformnost, avšak vyžaduje znalost jazyka Dart, který nebylo možné sdílet se serverovou částí. Nativní vývoj by poskytl maximální přístup k systémovým API za cenu nutnosti udržovat dvě zcela oddělené kódové základny.

Expo bylo zvoleno jako současný standard React Native vývoje. Managed workflow zjednodušuje konfiguraci nativních závislostí a nástroje `expo prebuild` a `expo run:*` umožňují generovat nativní projekty jak lokálně, tak v CI/CD pipeline na GitHub Actions. Expo SDK navíc poskytuje stabilní přístup k nativním modulům – včetně integrace s rozhraními Health Connect [5] a HealthKit [6].

1.3.2 Next.js pro webovou vrstvu

Doplňková webová vrstva systému byla implementována pomocí frameworku Next.js nad knihovnou React. Framework byl zvolen kvůli podpoře hybridního renderingu: veřejné landing page a stránka stahování mohou být generovány staticky s pravidelnou revalidací, zatímco administrátorské routy zůstávají dynamické a pracují se stavovou autentizací

založenou na cookies. Vestavěné route handlery současně umožňují obsluhovat stabilní URL pro nejnovější i archivní mobilní buildy bez potřeby samostatné distribuční služby.

1.3.3 Bun a Hono

Serverová část je postavena na rozhraní Bun [4] s využitím webového frameworku Hono [16]. Bun je moderní JavaScriptový runtime s výrazně vyšším výkonem oproti Node.js – v porovnání s Node.js dosahuje násobně kratších dob startu i vyšší propustnosti HTTP požadavků [17]. Klíčovou výhodou je vestavěná podpora jazyka TypeScript bez nutnosti transpilace, což zjednodušuje vývojový proces při zachování plné kompatibility s ekosystémem npm.

Hono je odlehčený webový framework optimalizovaný pro serverová prostředí. Vyznačuje se minimálními závislostmi, typově bezpečným směrováním požadavků a přímočarým API. Kombinace Bun + Hono byla zvolena jako moderní alternativa ke standardnímu stacku Node.js + Express.

1.3.4 PostgreSQL

Jako databázový systém byla zvolena databáze PostgreSQL [18]. Datový model aplikace LifeMeter zahrnuje vzájemně provázané entity (uživatelé, jídla, tréninky, spánkové záznamy), jejichž vztahy jsou přirozeně vyjádřeny relačním schématem s cizími klíči a referenční integritou. Relační databáze představuje přirozenější volbu oproti dokumentovým databázím (MongoDB) nebo embedded řešením (SQLite), která by nebyla vhodná pro serverovou část s souběžnými přístupy více uživatelů.

1.3.5 Better Auth

Pro správu autentizace a uživatelských relací byla zvolena knihovna Better Auth [19]. Tato volba byla upřednostněna před vlastní implementací i před cloudovými řešeními typu Firebase Authentication. Better Auth je navržen s nativní podporou jazyka TypeScript a poskytuje přímočarou integraci s frameworkem Hono prostřednictvím dedikovaného adaptéru. Jako self-hosted řešení nevyžaduje závislost na infrastruktuře třetí strany, čímž je zajištěna plná kontrola nad uživatelskými daty. Autentizace je realizována prostřednictvím `HttpOnly` cookies se stavovými relacemi na serveru, v souladu s doporučeními OWASP [10].

1.3.6 Docker a Oracle Cloud

Celá serverová a webová vrstva je kontejnerizována pomocí nástroje Docker Compose [20] a nasazen na instanci Oracle Cloud Infrastructure. Docker Compose umožňuje deklarativní definici vícekomponentního prostředí (API server, webový frontend a databáze) v jediném

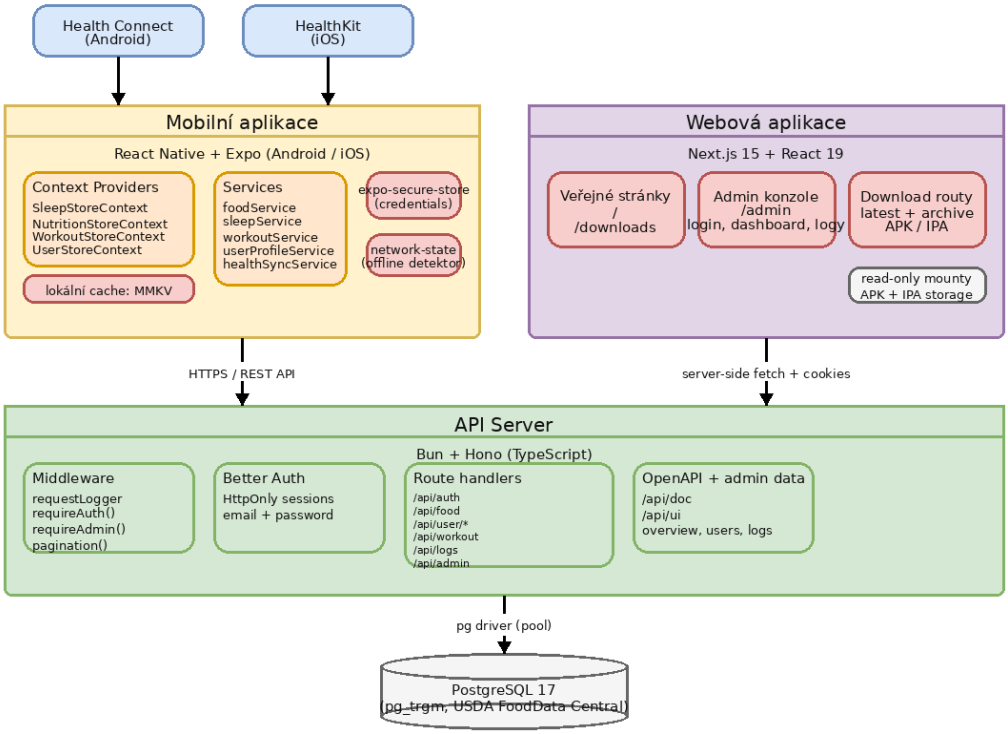
konfiguračním souboru, čímž zajišťuje reprodukovatelnost nasazení a jednoduchou správu závislostí mezi službami.

Oracle Cloud byl zvolen díky programu Always Free, který zahrnuje bezplatnou instanci s architekturou ARM s dostatečnými výpočetními prostředky pro provoz projektu. Na rozdíl od PaaS řešení poskytuje vlastní virtuální stroj plnou kontrolu nad konfigurací prostředí a správou uložených dat.

2. Metodika a vlastní řešení

2.1 Architektura systému

Systém LifeMeter se skládá ze tří hlavních součástí: mobilního klienta, webové vrstvy a serverové části nasazené v cloudovém prostředí. Mobilní i webový klient komunikují se serverem za použití REST API přes protokol HTTPS. Celkové uspořádání systému je znázorněno na obrázku 2.1.



Obrázek 2.1: Přehled architektury systému LifeMeter

2.1.1 Klientská vrstva

Mobilní aplikace je vyvinuta pomocí frameworku React Native ve spojení s platformou Expo [3], která zajišťuje přenositelnost mezi operačními systémy Android a iOS z jedné

sdílené kódové základny. Na platformě Android jsou zdravotní data čtena přes rozhraní Health Connect [5], na platformě iOS pak prostřednictvím HealthKit [6].

Aplikační stav je spravován čtveřicí doménových Context Providers – `SleepStoreContext`, `NutritionStoreContext`, `WorkoutStoreContext` a `UserStoreContext`. Každý provider zapouzdřuje data a operace pro příslušnou doménu a vystavuje je potomkům stromu komponent prostřednictvím React Context API. Lokální perzistenci dat zajišťuje knihovna MMKV [8]: data jsou primárně čtena z cache v paměti zařízení a synchronizována se serverem při obnovení síťového připojení, v souladu s principem *offline-first* [7]. Přihlašovací údaje jsou ukládány šifrovaně prostřednictvím `expo-secure-store`.

2.1.2 Webová vrstva

Webová část systému je implementována v adresáři `Web/` pomocí frameworku Next.js a slouží dvěma odlišným účelům. Veřejné routy `/` a `/downloads` fungují jako landing page a download centrum verzovaných mobilních buildů; jsou generovány staticky s krátkou periodou revalidace, aby bylo možné rychle obsloužit běžné návštěvy a současně promítnout nově publikované buildy. Druhou část tvoří administrátorská konzole pouze pro čtení pod cestou `/admin`, která po ověření role `admin` zobrazuje systémový přehled, logy a detail uživatelů. Route handlery `/downloads/[platform]/latest` a `/downloads/[platform]/[fileName]` zpřístupňují aktuální i archivní instalační soubory přímo z adresářů připojených do webového kontejneru.

2.1.3 Serverová vrstva

Serverová část je implementována v prostředí Bun s využitím webového frameworku Hono [16]. Každý příchozí požadavek prochází globálním middlewarovým řetězcem: `requestLogger` zaznamenává HTTP provoz, autentizační middleware injektuje informace o relaci (`c.get("user")`) a `pagination()` normalizuje parametry stránkování. Správu uživatelských relací zajišťuje knihovna Better Auth [19] prostřednictvím stavových `HttpOnly` cookies, v souladu s doporučeními OWASP [10].

Routy jsou organizovány doménově: `/api/auth` zpracovává přihlášení a registraci, `/api/food` poskytuje přístup k databázi potravin, `/api/user/*` sdružuje endpointy pro spánek, jídla, tréninky, uživatelský profil a synchronizaci zdravotních dat, `/api/workout` vystavuje referenční data (cviky, styly sérií), `/api/logs` systémové logy a `/api/admin` administrátorské přehledy nad uživatelskými daty. Chráněné routy vyžadují middleware `requireAuth()`, administrátorské pak `requireAdmin()`.

2.1.4 Datová vrstva

Datová vrstva je tvořena relační databází PostgreSQL 17 [18]. Rozšíření `pg_trgm` [21] umožňuje efektivní trigram-based fulltextové vyhledávání v databázi potravin, jejíž obsah pochází z veřejně dostupného zdroje USDA FoodData Central [22]. Serverová část komunikuje s databází prostřednictvím connection poolu knihovny `pg`.

2.2 Datový model

Databázové schéma aplikace LifeMeter je rozděleno do šesti doménových skupin: autentizace a správa relací (spravováno knihovnou Better Auth), výživa, spánek, tréninky, uživatelský profil a synchronizace zdravotních dat. Schéma je implementováno v PostgreSQL 17 a je verzováno prostřednictvím migračních SQL souborů. Entity-relationship diagram celého schématu je uveden na obrázku 2.2.

2.2.1 Autentizace

Tabulky pro správu uživatelů, relací a ověřování jsou generovány knihovnou Better Auth [19] a nejsou spravovány vlastními migracemi. Centrální entitou je tabulka `user` (viz tabulka 2.22 v příloze B), na kterou jsou pomocí cizích klíčů navázány všechny uživatelské záznamy v ostatních doménách.

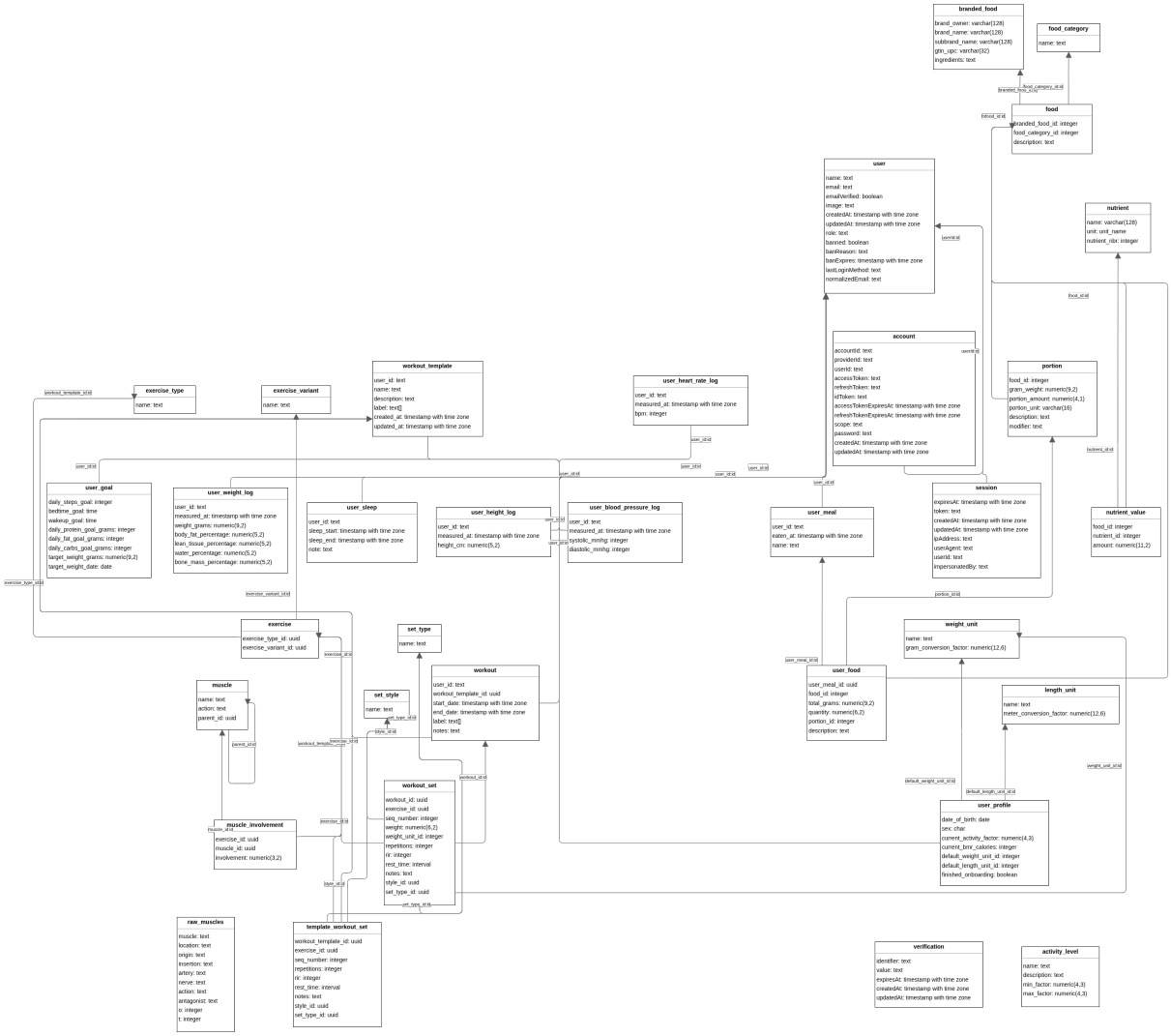
Dále jsou generovány tabulky `session` (aktivní relace s tokenem a IP adresou, viz tabulka 2.18), `account` (přihlašovací metoda a přihlašovací údaje, viz tabulka 2.3) a `verification` (jednorázové ověřovací tokeny, viz tabulka 2.35). Podrobné definice všech autentizačních tabulek jsou uvedeny v příloze B.

2.2.2 Výživa

Skupina výživových tabulek se skládá ze dvou částí: referenčních dat importovaných z databáze USDA FoodData Central [22] a uživatelských záznamů o konzumovaných jídlech. Struktura těchto tabulek (`food`, `nutrient_value`, `portion`, `user_meal` a `user_food`) je detailně popsána v příloze B (tabulky 2.9 až 2.24).

2.2.3 Spánek

Doména spánku zahrnuje tabulku `user_sleep` pro záznam denního spánku uživatele (viz tabulka 2.33 v příloze B).



Obrázek 2.2: Entity-relationship diagram databázového schématu

2.2.4 Tréninky

Doména tréninků se skládá z referenčních dat (typy cviků, svaly, styly a typy sérií) a uživatelských dat (šablony tréninků a zaznamenané tréninky). Definice hlavních tréninkových tabulek (`workout_template`, `template_workout_set`, `workout` a `workout_set`) jsou uvedeny v příloze B (tabulky 2.39 až 2.38).

Referenční tabulky `exercise` (kombinace typu a varianty cviku), `exercise_type`, `exercise_variant`, `muscle` (hierarchická tabulka svalů s vlastní rekurzivní vazbou `parent_id`), `muscle_involvement` (míra zapojení svalu v cviku), `set_style` a `set_type` obsahují pouze identifikátor a název, případně doplňující číselné atributy, a slouží jako číselníky (viz příloha B).

2.2.5 Uživatelský profil

Struktura profilu a cílů je uložena v tabulkách `user_profile` a `user_goal` (viz tabulky 2.32 a 2.25 v příloze B).

Dále jsou evidovány tabulky `user_weight_log` (hmotnost, viz tabulka 2.34), `user_height_log` (výška v cm s časovým razítkem), `user_blood_pressure_log` (systolický a diastolický tlak v mmHg) a `user_heart_rate_log` (tepová frekvence v bpm). Všechny protokolové tabulky sdílejí společný vzor: primární klíč `UUID`, cizí klíč `user_id` s `CASCADE DELETE` a časové razítko `measured_at` indexované sestupně pro efektivní výpis posledních hodnot. Kompletní definice těchto tabulek jsou k dispozici v příloze B.

2.2.6 Synchronizace zdravotních dat

Doména synchronizace zdravotních dat je reprezentována trojicí tabulek. Tabulka `user_health_sync_state` ukládá stav synchronizace pro jednotlivé providery (Health Connect a Apple Health). Tabulka `user_health_sync_pending_run` slouží k rozpracovaným dávkovým běhům. Tabulka `user_health_sync_source_item` mapuje zdrojové záznamy z externích platforem na cílové řádky v doménových tabulkách (`user_sleep`, `user_weight_log`, `user_height_log`, `user_heart_rate_log`, `user_blood_pressure_log`). Tato vrstva umožňuje idempotentní dávkovou synchronizaci, detekci smazaných záznamů a bezpečné obnovení vícebatchových běhů po přerušení. Detailní definice této trojice tabulek jsou uvedeny v příloze B (tabulky 2.28 až 2.27).

2.3 Popis řešení

2.3.1 Mobilní aplikace

Autentizace a správa relací

Autentizace je realizována prostřednictvím `AuthProvider` – kontextu obalujícího celý strom aplikace. Při inicializaci se volá `authService.getSession()`, jehož výsledek je uložen v zabezpečeném úložišti `expo-secure-store` pod klíčem `auth-session-cache`. Při selhání síťového volání je relace obnovena z tohoto lokálního záznamu, čímž je zachována funkčnost i při výpadku připojení. Přihlašování a registrace probíhá voláním REST endpointů Better Auth (`/api/auth/sign-in/email`, `/api/auth/sign-up/email`); stavová `HttpOnly` cookie je spravována serverem a automaticky přikládána ke každému požadavku prostřednictvím volby `credentials: "include"`.

Modul zdravotních dat

Čtení zdravotních dat ze systémových úložišť je zapouzdřeno do abstrakční vrstvy `src/lib/health/`, která sjednocuje rozhraní Health Connect [5] na platformě Android a HealthKit [6] na platformě iOS do jediného platformově neutrálního API. Každá funkce (například `getSteps`, `getSleep`, `getWeight`) vrací strukturu `Result<T>` rozlišující úspěšný výsledek a typovanou chybu, čímž je zamezeno nekontrolovanému šíření výjimek do vyšších vrstev.

Oprávnění jsou vyžádána jednorázově při startu aplikace funkcí `requestHealthPermissions()`. Na platformě Android je inicializován klient Health Connect a pro každý typ dat (`Steps`, `SleepSession`, `Weight` aj.) je vyžádáno oprávnění ke čtení; na platformě iOS je volána metoda `requestAuthorization()` knihovny HealthKit. Hodnoty spánkových fází jsou mapovány z číselných konstant příslušné platformy na sjednocenou množinu `"awake" | "core" | "deep" | "rem" | "asleep"`.

Lokální úložiště

Lokální perzistence dat je realizována modulem `src/lib/storage.ts`, který obaluje MMKV [8] typově bezpečnými pomocnými funkcemi pro čtení a zápis primitiv, objektů a polí serializovaných do JSON. Pro použití uvnitř komponent React jsou exportovány hooky `useStorage.string()`, `useStorage.boolean()`, `useStorage.object()` a `useStorage.array()`, jejichž změna hodnoty způsobí překreslení komponenty v souladu s modelem React. Pro imperativní přístup mimo komponenty (například ve frontě offline požadavků) slouží synchronní API objektu `storage`.

Offline fronta požadavků

Síťová vrstva aplikace (`src/lib/net.ts`) implementuje mechanismus odložení mutujících požadavků při výpadku připojení. Při každém volání funkce `request()` je nejprve zkontrolován příznak `isOffline` ze sdíleného modulu `network-state.ts`. Je-li zařízení offline, požadavek není odeslán, ale serializován a přiložen do fronty uložené v MMKV pod klíčem `offline-queue`. Duplicitní záznamy jsou odfiltrována porovnáním cesty a těla požadavku.

Stav sítě je monitorován komponentou `NetworkProvider` prostřednictvím funkce `addNetworkStateListener()` z knihovny `expo-network`. Při obnovení připojení je zavolána funkce `flushQueue()`, která sekvenčně přehraje uložené požadavky a neúspěšné ponechá ve frontě pro příští pokus. Souběžně je emitována událost `emitReconnect()`, na kterou jsou přihlášeny všechny doménové Context Providers a reagují opětovným načtením dat ze serveru. Podrobný popis algoritmů `request()` a `flushQueue()` je uveden v sekci 2.5.5.

Lokální oznámení

Plánování oznámení je centralizováno do hooku `useNotifications()` v souboru `src/lib/notifications.ts`. Funkce `scheduleNotification()` přijímá identifikátor, titulek, text, čas a příznak opakování. Opakující se oznámení (připomenutí spánku, vstávání, cvičení) jsou naplánována s triggerem typu `DAILY` zadáváním hodiny a minuty; jednorázová s triggerem `TIME_INTERVAL`. Stav naplánovaných oznámení je synchronizován s MMKV, aby byl dostupný i bez živého systémového volání. Při globálním vypnutí oznámení jsou zrušena všechna naplánovaná upozornění.

2.3.2 Serverová část

Middlewarový řetězec

Každý příchozí HTTP požadavek prochází middlewarovým řetězcem definovaným v `src/server.ts`. Jako první je aplikován middleware `secureHeaders()` nastavující bezpečnostní HTTP hlavičky (například `X-Content-Type-Options`, `X-Frame-Options`), následuje `trimTrailingSlash()` pro normalizaci cest URL a `requestLogger`, který zaznamenává metodu, cestu, stavový kód, dobu zpracování a IP adresu klienta. Na všech cestách je poté injektována informace o relaci voláním `auth.api.getSession()`: výsledek je uložen do kontextu jako `c.get("user")` a `c.get("session")`; pro nepřihlášené požadavky jsou obě hodnoty nastaveny na `null`. Chráněné routy uplatňují middleware `requireAuth()`, který vrací odpověď `401 Unauthorized` při chybějící relaci, resp. `requireAdmin()`, jenž navíc ověřuje roli `"admin"` v poli role uživatele.

Struktura endpointů

Routy jsou organizovány do šesti doménových skupin pod základní cestou `/api`. Skupina `/api/auth/*` je delegována přímo na handler knihovny Better Auth a nevyžaduje autentizaci. Skupina `/api/workout/*` vystavuje veřejně dostupná referenční data (cviky, styly a typy sérií, váhové možnosti). Skupina `/api/food` poskytuje stránkovaný seznam potravin a fulltextové vyhledávání (parametr `name` využívající `pg_trgm` [21], parametr `gtin` pro vyhledávání dle čárového kódu). Skupina `/api/user/*` sdružuje chráněné endpointy pro spánek (`/sleep`), výživu (`/food`), tréninky (`/workout`, `/workout/template`), uživatelský profil (`/data`) a synchronizaci zdravotních dat (`/sync`). Zbývající skupiny `/api/logs` a `/api/admin/*` obsluhují administrátorské logy a dohled nad uživatelskými daty pouze pro čtení. Schémata požadavků a odpovědí jsou validována knihovnou Zod a dokumentována prostřednictvím integrace `@hono/zod-openapi`; výsledná OpenAPI dokumentace je přístupná na `/api/doc` a `/api/ui`.

Nasazení a reverzní proxy

Serverová a webová část jsou provozovány na instanci Oracle Cloud Infrastructure s architekturou ARM. Kontejnerizační soubor `docker-compose.yml` definuje tři služby: `db` (PostgreSQL 17 s pojmenovaným svazkem `pgdata` pro perzistenci dat), `api` (sestavení ze zdrojového kódu adresáře `Server/`, mapování portu 3000 a sdílený svazek pro výstup logovacích souborů) a `web` (Next.js frontend z adresáře `Web/`, publikovaný interně na portu 3001). Webový kontejner má navíc mounty pouze pro čtení adresářů `releases/apk` a `releases/ios`, odkud obsluhuje aktuální i archivní instalační soubory pro download centrum. Docker Compose [20] běží přímo na VM; reverzní proxy Caddy je nainstalována odděleně jako systémová služba a spravuje ukončení TLS pro domény `lifemeter.fit` a `www.lifemeter.fit`.

Caddy je nakonfigurována se dvěma pravidly omezení frekvence požadavků (*rate limiting*) rozlišujícími dvě časová okna: maximálně 10 požadavků za sekundu a 100 požadavků za minutu, obojí klíčované IP adresou klienta (`key {remote_host}`). Požadavky na cestu `/api` a její podstromy jsou předávány na interní port 3000 aplikačního kontejneru, zatímco veškeré ostatní požadavky jsou směrovány na webový frontend na portu 3001. Tím jsou veřejné stránky, download centrum i admin konzole obslouženy přes stejnou doménu jako samotné API.

2.3.3 CI/CD pipeline

Průběžné nasazení serverové a webové části je zajištěno workflow GitHub Actions (`.github/workflows/deploy.yml`). Workflow se spouští automaticky při každém přímém commitu nebo sloučení pull requestu do větve `main`, pokud změna zasahuje do adresářů

`Server/` nebo `Web/`, případně do `docker-compose.yml`. Jediný krok *Deploy to Oracle VM* využívá akce `appleboy/ssh-action`, která se připojí na produkční server prostřednictvím SSH klíče uloženého v tajných proměnných repozitáře (`SSH_HOST`, `SSH_USER`, `SSH_KEY`). Na serveru jsou poté provedeny příkazy `git pull origin main`, `docker compose build -pull` (s fallbackem na build bez cache při selhání) a `docker compose up -d -remove-orphans`, čímž dochází k nasazení aktualizované serverové i webové vrstvy s minimálním výpadkem. Databáze zůstává v provozu díky pojmenovanému svazku `pgdata`, který není součástí životního cyklu kontejnerů.

Mobilní klient je publikován samostatným workflow `.github/workflows/app-apk.yml`. Při změně adresáře `App/` se v GitHub Actions provede `npm ci`, `expo prebuild` pro Android, sestavení release APK pomocí Gradle a následné nahrání výsledného artefaktu na Oracle VM do adresáře `releases/apk`. Současně jsou aktualizována metadata `latest.json`, která využívá webová stránka `/downloads` pro zobrazení nejnovějšího sestavení a jeho verzovaného archivu.

2.4 Návrh aplikace

Návrh aplikace LifeMeter byl proveden před zahájením implementace a sloužil jako závazný rámec pro vývoj. Výsledkem návrhu jsou tři vzájemně doplňující se modely: diagram případů užití popisující funkcionalitu z pohledu uživatele, diagram navigace zachycující strukturu obrazovek a přechody mezi nimi a diagram tříd dokumentující architekturu doménových kontextů klientské vrstvy.

2.4.1 Případy užití

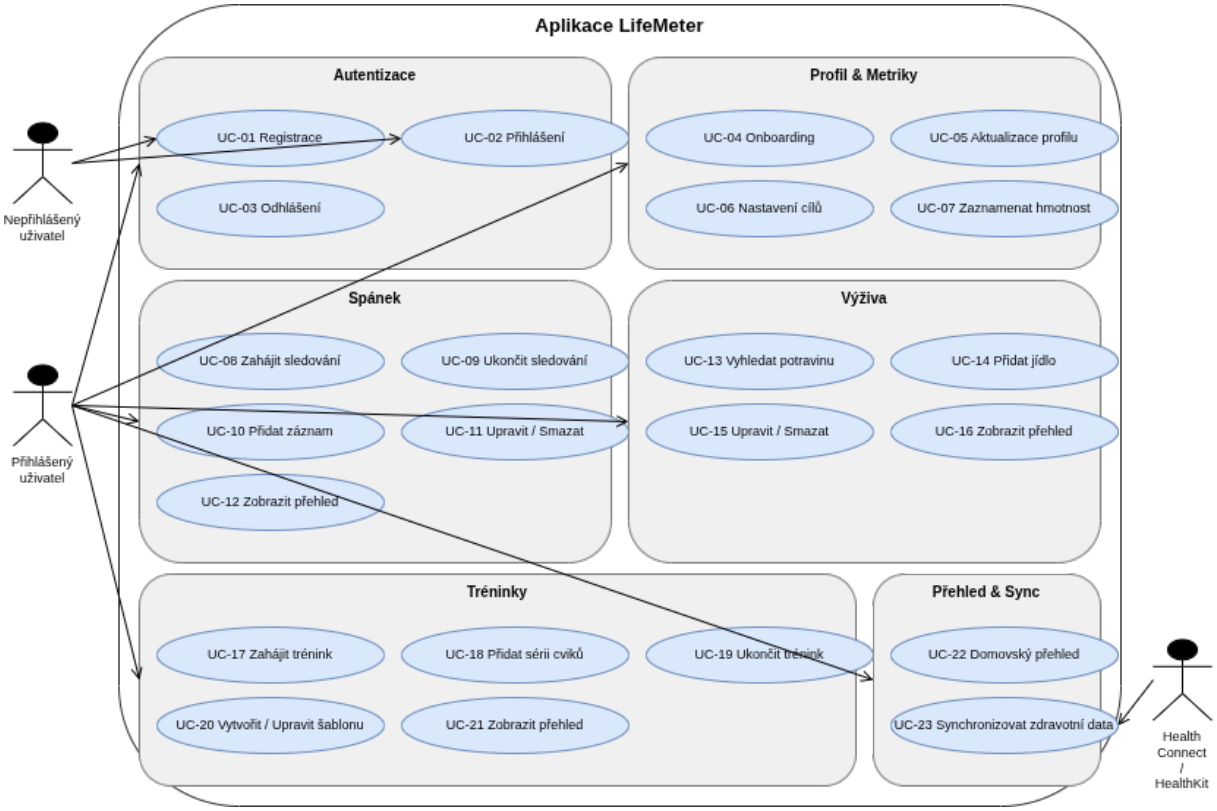
V systému LifeMeter jsou definováni tři aktéři. *Nepřihlášený uživatel* může provést registraci nebo přihlášení. *Přihlášený uživatel* má přístup ke všem funkcím aplikace včetně správy profilu, záznamu spánku, výživy a tréninků a zobrazení domovského přehledu. Externím aktérem je systémová platforma *Health Connect* (Android) resp. *HealthKit* (iOS), se kterou aplikace synchronizuje zdravotní data.

Identifikované případy užití jsou znázorněny na obrázku 2.3

2.4.2 Diagram navigace

Navigační struktura aplikace je rozdělena do dvou zásobníků obrazovek (*stack navigátorů*), jejichž zobrazení závisí na stavu autentizace a existenci uživatelského profilu (obrázek 2.4).

OnboardingStack je prezentován nepřihlášeným uživatelům a obsahuje uvítací obrazovku (*Title*), formulář přihlášení (*SignIn*) a registrace (*SignUp*). Po úspěšné autentizaci je



Obrázek 2.3: Diagram případů užití aplikace LifeMeter

zkontrolována existence profilu: pokud profil neexistuje nebo nebyl dokončen onboarding, je uživatel přesměrován na obrazovku *OnboardingInfoScreen*.

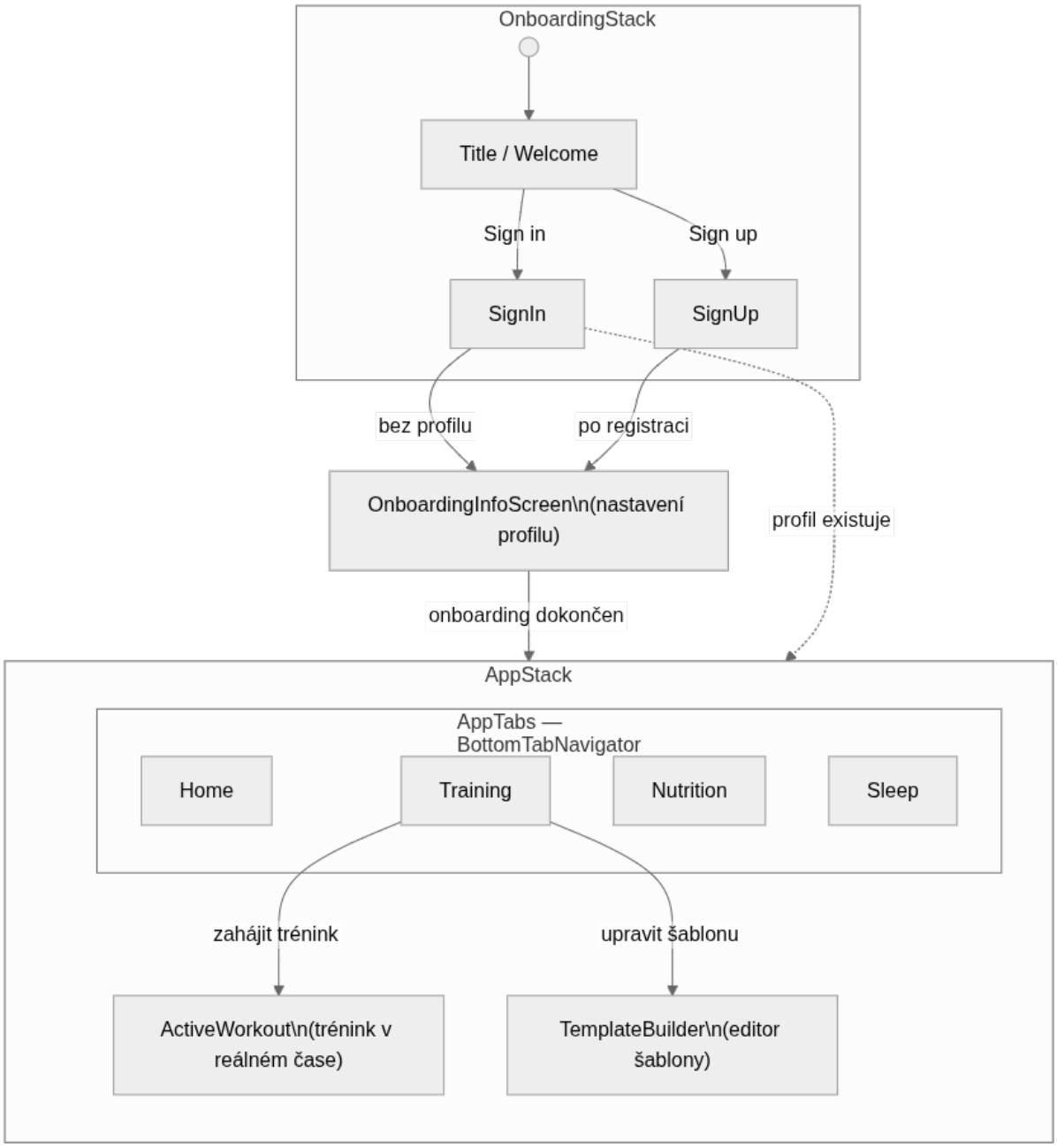
AppStack je zpřístupněn přihlášeným uživatelům s dokončeným profilem. Výchozím prvkem je *AppTabs* – zásobník se spodní navigační lištou obsahující čtyři záložky: *Home*, *Training*, *Nutrition* a *Sleep*. Nad tímto zásobníkem jsou nadřazeny dvě modální obrazovky: *ActiveWorkout* (aktivní trénink v reálném čase) a *TemplateBuilder* (editor šablony tréninku), přístupné z obrazovky *Training*.

Správa přihlašovacího stavu je zajišťována komponentou `AuthProvider`, která obaluje celý strom aplikace a poskytuje informaci o aktuálním uživateli prostřednictvím React Context API.

2.4.3 Diagram tříd

Správa aplikačního stavu klientské vrstvy je realizována čtveřicí doménových rozhraní (*Context Providers*), z nichž každý zapouzdřuje data a operace jedné funkční domény (obrázek 2.5).

`SleepStoreContext` spravuje kolekci spánkových záznamů a zpřístupňuje operace pro jejich vytváření, editaci a mazání včetně detekce probíhající spánkové relace (`ongoingSleepSession`).



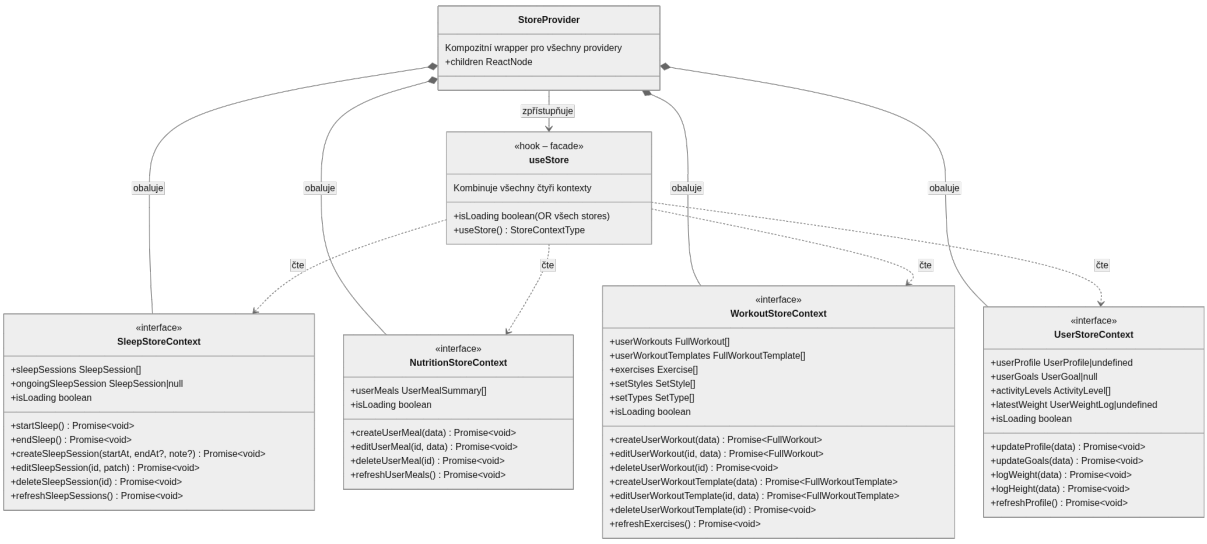
Obrázek 2.4: Diagram navigační struktury aplikace LifeMeter

`NutritionStoreContext` uchovává jídelní záznamy ve struktuře jako dvojici objektu `userMeal` a pole `userFoods`; nad oběma částmi poskytuje operace CRUD pro jídla a jídelní deníky.

`WorkoutStoreContext` spravuje zaznamenané tréninky, šablony tréninků a referenční číselníky (cviky, styly a typy sérií). Jeho rozhraní je nejrozsáhlejší a odráží komplexnost tréninkové domény.

`UserStoreContext` poskytuje přístup k uživatelskému profilu, stanoveným cílům a protokolárním záznamům tělesných metrik (hmotnost, výška).

Všechny čtyři providery jsou kompozičně zahrnuty v komponentě `StoreProvider`, která je vložena do kořene stromu aplikace. Pro zpětnou kompatibilitu a pohodlné použití v místech, kde je potřeba přistupovat k více doménám současně, je k dispozici fasádní hook `useStore()`, který sloučí výstupy všech čtyř kontextů do jediné struktury `StoreContextType`. Jeho použití však způsobuje překreslení komponenty při změně v *jakékoliv* doméně; z toho důvodu je doporučeno v nových komponentách preferovat granulární hooky `useSleepStore()`, `useNutritionStore()`, `useWorkoutStore()` a `useUserStore()`.



Obrázek 2.5: Diagram tříd – doménové Context Providers klientské vrstvy

2.5 Popis důležitých algoritmů

2.5.1 Výpočet metabolismu a denního energetického výdeje

Bazální metabolismus (*Basal Metabolic Rate*, BMR) vyjadřuje minimální množství energie potřebné k udržení základních životních funkcí organismu v klidovém stavu. V aplikaci

LifeMeter je BMR vypočítán rovnicí Mifflin–St. Jeor [23], která je v současné klinické praxi považována za nejpřesnější predikci klidového energetického výdeje pro obecnou populaci.

Pro pohlaví *muž* platí:

$$\mathrm{BMR}_{\text{muž}} = 10 \cdot m + 6{,}25 \cdot h - 5 \cdot a + 5 \tag{2.1}$$

Pro pohlaví *žena* platí:

$$\mathrm{BMR}_{\text{žena}} = 10 \cdot m + 6{,}25 \cdot h - 5 \cdot a - 161 \tag{2.2}$$

kde m je tělesná hmotnost v kilogramech, h je výška v centimetrech a a je věk v letech. Vstupní hodnoty zadané v jednotkách lbs, st nebo ft jsou před dosazením převedeny na soustavu SI.

Celkový denní energetický výdej (*Total Daily Energy Expenditure*, TDEE) je získán vynásobením BMR koeficientem fyzické aktivity f:

$$\mathrm{TDEE} = \mathrm{BMR} \cdot f \tag{2.3}$$

Koeficient f je uživatelem volen při onboardingu na základě úrovně pohybové aktivity; každá úroveň je definována rozsahem $[f_{\min}, f_{\max}]$ a ve výpočtu je použita střední hodnota $f = (f_{\min} + f_{\max})/2$.

2.5.2 Stanovení cílového denního kalorického příjmu

Pokud uživatel v průvodci nastavením zadá cílovou hmotnost a cílové datum, je vypočítán upravený denní kalorický příjem tak, aby bylo cíle dosaženo v požadovaném čase. Výpočet vychází z energetické ekvivalence tukové tkáně, kde změna 1 kg tělesné hmotnosti odpovídá přibližně 7 700 kcal [24].

Celková kalorická bilance potřebná ke změně hmotnosti o Δm kilogramů za d dní:

$$\Delta E = \Delta m \cdot 7700 \quad [\text{kcal}] \tag{2.4}$$

Denní kalorická korekce:

$$\delta = \frac{\Delta E}{d} \tag{2.5}$$

Cílový denní příjem:

$$C_{\text{cíl}} = \mathrm{TDEE} + \delta \tag{2.6}$$

Hodnota $\delta > 0$ odpovídá kalorickému přebytku (nabírání hmotnosti), $\delta < 0$ kalorickému deficitu (hubnutí). Pokud uživatel cílovou hmotnost nezadá, je jako cílový příjem použito přímo TDEE.

2.5.3 Výpočet nutričních hodnot jídla

Každá potravina v databázi USDA FoodData Central [22] obsahuje nutriční hodnoty vztažené na 100 g jedlého podílu. Při záznamu jídla uživatel zadá skutečnou gramáž porce g; obsah živiny N pro danou potravinu je pak škálován proporčním faktorem:

$$N_{\text{porce}} = N_{100} \cdot \frac{g}{100} \tag{2.7}$$

Celkový kalorický obsah jídla je stanoven na základě Atwaterových energetických koeficientů [25]:

$$E = 4 \cdot P + 4 \cdot S + 9 \cdot T \quad [\text{kcal}] \tag{2.8}$$

kde P je obsah bílkovin, S obsah sacharidů a T obsah tuků v gramech. Živiny jsou identifikovány prostřednictvím standardizovaných kódů USDA: nutrient_nbr 208 (energie), 203 (bílkoviny), 205 (sacharidy), 204 (tuky).

Pokud uživatel nestanoví explicitní cíle pro jednotlivé makronutrienty, jsou tyto automaticky odvozeny z TDEE s výchozím rozložením 30 % bílkoviny, 40 % sacharidy a 30 % tuky:

$$P_{\text{cíl}} = \left\lfloor \frac{\text{TDEE} \cdot 0{,}30}{4} \right\rfloor \tag{2.9}$$

$$S_{\text{cíl}} = \left\lfloor \frac{\text{TDEE} \cdot 0{,}40}{4} \right\rfloor \tag{2.10}$$

$$T_{\text{cíl}} = \left\lfloor \frac{\text{TDEE} \cdot 0{,}30}{9} \right\rfloor \tag{2.11}$$

2.5.4 Fulltextové vyhledávání potravin

Databáze potravin čítá přes milion záznamů z datového zdroje USDA FoodData Central [22]. Efektivní vyhledávání je zajištěno kombinací PostgreSQL full-text search a GIN indexu. Sloupec `search_vector` je persistován jako generovaný sloupcový `tsvector`:

```
ALTER TABLE food
  ADD COLUMN search_vector tsvector
  GENERATED ALWAYS AS (to_tsvector('english', description)) STORED;

CREATE INDEX idx_food_search_vector
  ON food USING GIN (search_vector);
```

Listing 2.1: Definice generovaného sloupce a GIN indexu

Vstupní dotaz je na straně serveru sanitizován – jsou odstraněny speciální znaky operátorů PostgreSQL (`&`, `|`, `!`, závorky apod.) a každý token je rozšířen o prefixový operátor `:*`, který zajišťuje, že dotaz „appl“ nalezne i záznamy obsahující „apple“ nebo „applesauce“. Jednotlivé tokeny jsou spojeny operátorem AND:

```
vstup:  "whole grain bread"
vystup: "whole:* & grain:* & bread:*"
```

Listing 2.2: Transformace vstupního řetězce na tsquery

Výsledky jsou řazeny víceúrovňovým klíčem tak, aby byly upřednostněny stručné a přesné shody před delšími nebo méně relevantními záznamy:

1. přesná shoda s dotazem (`ILIKE $query`) – priorita 0,
2. prefixová shoda (`ILIKE $query||'%'`) – priorita 1,
3. ostatní výsledky seřazené dle `ts_rank_cd` a délky popisu – priorita 2.

2.5.5 Offline fronta požadavků

Aplikace implementuje princip *offline-first* [7]: při ztrátě síťového připojení nejsou mutační požadavky (POST, PUT, DELETE) zahozeny, nýbrž jsou přidány do fronty v lokálním úložišti MMKV [8] a odeslány po obnovení připojení. Čtecí požadavky (GET, HEAD) jsou vždy odesílány okamžitě a do fronty zařazeny nejsou.

```
FUNCTION request(path, init) -> T | undefined
  IF isOffline AND method(init) ≠ GET AND offlineModeEnabled THEN
    IF NOT queue.contains(path, init) THEN   -- deduplikace
      queue.push(path, init)                 -- persistuj do MMKV
    END IF
    RETURN undefined                         -- optimisticky navrat
  END IF

  controller ← AbortController(timeout = 5000 ms)
  response   ← HTTP.send(path, init, signal = controller)
  RETURN parse(response)
END FUNCTION

FUNCTION flushQueue() -> void
  pending ← queue.readAll()
  queue.clear()                              -- smaz pred odesilani

  FOR EACH (path, init) IN pending DO
```

```
    IF request(path, init) FAILS THEN
      queue.push(path, init)                -- vrat zpet pri selhani
    END IF
  END FOR
END FUNCTION
```

Listing 2.3: Pseudokód zpracování HTTP požadavku s offline frontou

Deduplikace je zajištěna porovnáním serializované cesty a těla požadavku před vložením do fronty, čímž je zamezeno opakovanému záznamu totožné operace.

Každý HTTP požadavek je opatřen timeoutem 5 000 ms prostřednictvím `Abort Controller`, čímž je zamezeno blokování při nedostupném serveru.

Výsledky

Výsledkem projektu je funkční mobilní aplikace LifeMeter dostupná na operačních systémech Android (verze 10 a vyšší), doplněná o serverový backend a webovou vrstvu nasazené na doméně `lifemeter.fit`. Webová část poskytuje veřejnou prezentaci, verzované centrum stahování mobilních buildů a administrátorskou konzoli pouze pro čtení.

2.6 Splnění požadavků zadání

Následující tabulka uvádí přehled všech požadavků vymezených v zadání maturitního projektu a jejich stav po dokončení implementace.

Tabulka 2.1: Přehled plnění požadavků zadání

Požadavek	**Stav**	**Poznámka**
Záznam fyzických aktivit	splněno	Aktivní tréninky se sadami, opakováními a váhou; tréninkové šablony s přetahováním cvičení.
Přidávání potravin, sledování kalorií a makroživin	splněno	Textové vyhledávání i skenování čárového kódu; přehled bílkovin, sacharidů a tuků za aktuální den.
Sledování mikroživin a vitamínů	splněno	Přehled všech mikronutrientů (vitamíny, minerály) dostupný přes rozbalovací panel.
Záznam délky spánku s poznámkou	splněno	Ruční zadání spánku i spuštění/ukončení v reálném čase; volitelná textová poznámka ke každému záznamu.
Přihlašování e-mailem se zabezpečeným ověřením	splněno	Registrace a přihlášení prostřednictvím knihovny Better Auth; relace na bázi bezpečných HTTP cookies.
Offline režim se synchronizací	splněno	Mutační požadavky jsou při absenci připojení ukládány do fronty MMKV a odeslány po jeho obnovení.
Notifikace ke cvičení a spánku	splněno	Konfigurovatelná denní upozornění na čas cvičení, dobu uložení ke spánku a čas vstávání.
Integrace s jinou aplikací/platformou	splněno	Health Connect (Android) a Apple HealthKit (iOS): kroky, spánek, váha, výška, spálené kalorie.

2.7 Implementované moduly

Aplikace je rozdělena do pěti hlavních funkčních modulů přístupných přes spodní navigační lištu.

Onboarding provádí nového uživatele čtyřkrokovým průvodcem: zadání základních údajů (pohlaví, datum narození, preferované jednotky), tělesných měr (výška, váha, případně procento tělesného tuku), úrovně pohybové aktivity a cílů (cílová váha, datum dosažení, denní kroky, časy spánku). Z těchto vstupů je vypočítán bazální metabolismus metodou Mifflin–St Jeor a z něj odvozeny personalizované makronutrientní cíle.

Domovská obrazovka zobrazuje přehledový dashboard za posledních 7 dní: sloupcový graf denních kroků synchronizovaný z platformy zdravotních dat, kalorický graf výživy, přehled doby spánku a graf tréninkové aktivity.

Modul výživy umožňuje přidávání jídel textovým vyhledáváním v databázi přes 1 000 000 potravin ze zdroje USDA FoodData Central nebo skenováním čárového kódu EAN-13/EAN-8/UPC. Denní makra jsou zobrazena graficky jako okruháčové grafy s ab-

solutními hodnotami a procentuálním plněním cíle. Přehled mikronutrientů je dostupný prostřednictvím vysunovacího panelu.

Modul spánku uchovává historii spánkových záznamů a umožňuje spuštění spánkové relace v reálném čase nebo ruční zadání časového rozmezí. Na zařízeních se zdravotní platformou jsou automaticky importovány historické záznamy spánkových fází (bdění, lehký spánek, hluboký spánek, REM).

Modul tréninku nabízí správu tréninkových šablon s možností přetahování pořadí cvičení a zaznamenávání aktivního tréninku s běžícím časovačem. Grafy průběhu výkonu u jednotlivých cvičení sledují maximální váhu, odhadnuté maximum pro jedno opakování (1RM) dle Epleyho vzorce a celkový tréninkový objem.

Nastavení (přístupné z hlavičky aplikace) sdružuje správu profilu a cílů, konfiguraci oznámení, přepínač offline režimu, propojení s platformou zdravotních dat a přepínač světlého a tmavého motivu.

Mimo samotnou mobilní aplikaci byla implementována také **veřejná webová vrstva**, která prezentuje produkt, zpřístupňuje aktuální i archivní mobilní buildy a sjednocuje distribuční odkazy. Na stejné doméně běží rovněž **administrátorská konzole pouze pro čtení**, určená pro kontrolu systémového přehledu, logů a detailů uživatelských účtů.

2.8 Technické parametry

Tabulka 2.2: Přehled technických parametrů aplikace

Parametr	**Hodnota**
Podporované platformy	Android 10+, iOS 16+
Počet obrazovek (navigačních uzlů)	10 (4 hlavní záložky, onboarding, aktivní trénink, sestavení šablony, seznam spánku, přihlášení, registrace)
Počet webových rout	8 (landing page, download centrum, admin login, forbidden, dashboard, logy, uživatelé, detail uživatele)
Počet REST API endpointů	52 explicitně definovaných + `/api/auth/*`
Počet databázových tabulek	37 dokumentovaných tabulek (33 aplikačních + 4 autentizační Better Auth)
Počet potravin v databázi	přes 1 000 000 (zdroj USDA FoodData Central, CC0 1.0)
Počet sledovaných nutrientů na potravinu	až 150 (makronutrienty, vitamíny, minerály)
Fulltextový index vyhledávání	PostgreSQL `tsvector` + GIN index + pg_trgm
Prostředí nasazení	Oracle Cloud Infrastructure (ARM), Docker Compose (`db`, `api`, `web`), Caddy reverzní proxy, TLS
Průběžné nasazení	GitHub Actions (`deploy.yml` pro stack, `app-apk.yml` pro vydání APK)
Zdravotní integrace	Health Connect (Android), Apple HealthKit (iOS)
Sledované zdravotní metriky	kroky, spánek (fáze), váha, výška, spálené kalorie

2.9 Uživatelské rozhraní

Následující snímky obrazovek dokumentují výslednou podobu aplikace. Aplikace podporuje světlý i tmavý motiv; zobrazené snímky zachycují tmavý motiv.

[snímek obrazovky: onboarding.png]

Obrázek 2.6: Uvítací obrazovka onboardingu

[snímek obrazovky: home.png]

Obrázek 2.7: Domovská obrazovka – 7denní přehled

[snímek obrazovky: nutrition.png]

Obrázek 2.8: Modul výživy – denní makra a přehled jídel

[snímek obrazovky: food-search.png]

Obrázek 2.9: Vyhledávání potravin a skenování čárového kódu

[snímek obrazovky: sleep.png]

Obrázek 2.10: Modul spánku – přehled a záznam

[snímek obrazovky: training.png]

Obrázek 2.11: Modul tréninku – aktivní trénink se sadami

[snímek obrazovky: template-builder.png]

Obrázek 2.12: Sestavení tréninkové šablony

[snímek obrazovky: settings.png]

Obrázek 2.13: Nastavení – notifikace a zdravotní integrace

2.10 Testování

Aplikace byla průběžně testována ručně na fyzickém zařízení se systémem Android 14 (Samsung Galaxy S23). Testování se zaměřilo na správnost ukládání a zobrazování dat ve všech třech sledovaných doménách (výživa, spánek, trénink), funkčnost offline fronty při vypnutém datovém připojení, správné odeslání zařazených požadavků po jeho obnovení a správnost výpočtů makronutrientních cílů.

Dále bylo provedeno uživatelské testování s testovacími uživateli, kteří aplikaci používali po dobu několika dní v běžném provozu. Na základě jejich zpětné vazby byly provedeny úpravy uživatelského rozhraní – zejména zjednodušení průvodce onboardingem a přidání možnosti přeskočit volitelné položky tělesné kompozice.

Diskuze

2.11 Zhodnocení dosažených výsledků

Aplikace LifeMeter splňuje všechny požadavky vymezené v zadání maturitního projektu a v několika oblastech zadání překračuje. Zatímco zadání specifikuje podporu systému Android 10 a vyšší, výsledná aplikace je funkční rovněž na platformě iOS díky sdílené kódové základně v jazyce TypeScript. Integrace s jinou aplikací, požadovaná v zadání v jednotném čísle, byla realizována souběžně pro obě platformy: prostřednictvím rozhraní Health Connect na Androidu a Apple HealthKit na iOS. Serverová část, jejíž existence ze zadání přímo nevyplývala, byla vyvinuta jako plnohodnotný systém zahrnující REST API s 52 explicitně definovanými endpointy, veřejnou webovou prezentaci v Next.js, administrátorskou konzoli pouze pro čtení, dokumentaci generovanou ve standardu OpenAPI a produkční nasazení na cloudové infrastruktuře s automatickým průběžným nasazením.

2.12 Zdůvodnění volby technologií

2.12.1 React Native a Expo

Volba frameworku React Native namísto nativního vývoje v jazycích Swift/Kotlin byla motivována požadavkem na podporu obou mobilních platforem při zachování jedné kódové základny. Alternativou byl framework Flutter, avšak jeho ekosystém zdravotních integrací (Health Connect, HealthKit) byl v době zahájení vývoje méně stabilní než dostupná řešení pro React Native. Platforma Expo doplňuje React Native o standardizovaný build systém a přístup k nativním API prostřednictvím typovaných modulů; použití samotného React Native bez Expo by vyžadovalo ruční správu nativních projektů Xcode a Android Gradle, což by projekt prodražilo na čase bez přidané funkční hodnoty.

2.12.2 Bun a Hono

Pro serverovou část byl zvolen runtime Bun namísto zavedených alternativ Node.js nebo Deno. Bun dosahuje výrazně vyšší propustnosti při zpracování HTTP požadavků a dispo-

nuje vestavěným správcem balíčků, testovacím prostředím a bundlerem, čímž redukuje závislost na nástrojích třetích stran. Webový framework Hono byl upřednostněn před Express.js pro svůj typově bezpečný router a nativní integraci s knihovnou Zod pro validaci schémat a generování OpenAPI dokumentace. Tato volba umožnila udržovat dokumentaci API v synchronizaci se zdrojovým kódem bez nutnosti samostatných nástrojů.

2.12.3 PostgreSQL a volba relační databáze

Pro perzistenci dat byla zvolena relační databáze PostgreSQL namísto dokumentové databáze (např. MongoDB). Datový model aplikace – jídla, porce, živiny, uživatelské záznamy – je přirozeně relační a JOIN operace přes normalizované tabulky jsou pro tento typ dotazů výkonnější než vyhledávání v zanořených dokumentech. PostgreSQL navíc poskytuje nativní podporu pro fulltextové vyhledávání (`tsvector`, GIN indexy) a rozšíření `pg_trgm`, které byly klíčové pro implementaci vyhledávání v databázi přes milion potravin.

2.12.4 Better Auth a správa relací

Autentizace uživatelů je realizována knihovnou Better Auth s relacemi uloženými v `HttpOnly` cookies namísto tokenů JWT. Tato volba vychází z doporučení OWASP [10]: token JWT uložený v dostupném úložišti mobilního zařízení (např. `AsyncStorage`) je zranitelný vůči útokům XSS a krádeži tokenu. Relace spravovaná serverem a identifikovaná cookie, která není přístupná JavaScriptovému kódu, tento vektor útoku eliminuje. Nevýhodou stavových relací je náročnější horizontální škálování (nutnost sdíleného úložiště relací), která však pro projekt tohoto rozsahu nepředstavuje praktické omezení.

2.12.5 MMKV pro offline frontu

Pro implementaci offline fronty bylo zvoleno úložiště MMKV namísto standardní alternativy `AsyncStorage`. MMKV je implementováno jako sdílená paměť mapovaná na soubor; v benchmarcích dosahuje přibližně 10–30násobně vyšší rychlosti čtení a zápisu oproti `AsyncStorage` [8]. Pro frontu požadavků, do které jsou zapisovány i čteny záznamy při každé síťové operaci, je tato výkonnostní výhoda znatelná zejména u zařízení střední třídy.

2.13 Srovnání s existujícími řešeními

Aplikace LifeMeter je srovnávána s řešeními analyzovanými v teoretické části práce.

Oproti Google Fit [11] a Apple Health [12], které plní primárně roli datových agregátorů a nenabízejí vlastní rozhraní pro zadávání výživy nebo silových tréninků, LifeMeter poskytuje kompletní pracovní postup od zadání dat po vizualizaci trendů v rámci jedné

aplikace. Obě platformy jsou zároveň využity jako zdroj dat (kroky, spánek, tělesná hmotnost) prostřednictvím jejich zdravotních rozhraní.

MyFitnessPal [13] disponuje výrazně rozsáhlejší komunitně budovanou databází potravin a skenováním čárových kódů v globálním měřítku. LifeMeter oproti tomu vychází výhradně z ověřené vědecké databáze USDA FoodData Central [22] s licencí CC0 1.0, čímž je zaručena konzistentní nutriční přesnost dat bez závislosti na správnosti komunitních příspěvků. Sledování silových tréninků je v MyFitnessPal omezeno na odhad kalorického výdeje; LifeMeter zaznamenává kompletní strukturu tréninku (sady, opakování, váha, typ sady, čas odpočinku) a sleduje individuální výkonnostní trend pro každé cvičení.

Cronometer [14] se od LifeMeteru liší specializací na nutriční přesnost a sledování mikronutrientů s vazbou na vědecké databáze. LifeMeter mikronutrienty sleduje rovněž, avšak jeho záběr je šíříší – integruje silový trénink a spánkové záznamy, které Cronometer v základní verzi nenabízí.

Vůči aplikaci Sleep Cycle [15], která se specializuje na analýzu spánkových fází pomocí mikrofonu nebo akcelerometru, LifeMeter nenabízí automatickou detekci spánkových fází přes senzory zařízení; fáze spánku jsou importovány z platformy zdravotních dat, pokud je uživatel zaznamenává jiným zařízením (chytré hodinky apod.). Výhodou LifeMeteru je integrace spánkových dat s výživovými a tréninkovými záznamy v jednom přehledu.

2.14 Omezení implementace

V průběhu vývoje byla identifikována následující omezení:

Databáze potravin USDA FoodData Central obsahuje primárně americké potraviny a značkové výrobky distribuované na americkém trhu. Dostupnost evropských nebo českých potravin a zejména privátních značek tuzemských řetězců je omezená. Rozšíření databáze o lokalizovaná data by vyžadovalo integraci dalšího datového zdroje nebo vytvoření mechanismu pro přidávání vlastních potravin uživatelem.

Automatická detekce spánkových fází bez externího zařízení (chytrých hodinek) implementována není. Záznamy spánkových fází jsou importovány z platformy zdravotních dat, jejichž přítomnost závisí na vybavení uživatele.

2.15 Možná budoucí rozšíření

Na základě zpětné vazby z uživatelského testování a identifikovaných omezení jsou jako nejvýznamnější potenciální rozšíření projektu považována:

Přidání možnosti vytvářet a upravovat vlastní potraviny uživatelem, čímž by bylo překonáno omezení v pokrytí lokálních potravin databází USDA.

Implementace widgetů pro domovskou obrazovku operačního systému, které by umožnily rychlý přehled kalorického příjmu nebo denních kroků bez nutnosti otevírat aplikaci.

Rozšíření modulu spánku o automatické sledování prostřednictvím senzoru akcelerometru zařízení jako alternativu k importu z chytrých hodinek.

Integrace s platebními nebo předplatitelskými modely by v případě komerčního nasazení umožnila financování provozních nákladů serverové infrastruktury, které v současné verzi nese autor projektu.

Rozšíření na platformu Wear OS nebo watchOS by umožnilo zaznamenávat trénink přímo z chytrých hodinek bez nutnosti interakce s telefonem.

Závěr

Cílem maturitního projektu bylo navrhnout a implementovat mobilní aplikaci LifeMeter umožňující centralizované sledování fyzické aktivity, výživy a spánku. Všechny požadavky vymezené v zadání byly splněny; výsledná aplikace v několika ohledech zadání překračuje. Vedle primárně požadované platformy Android 10+ je aplikace funkční rovněž na iOS díky sdílené kódové základně v jazyce TypeScript. Integrace se systémovými zdravotními platformami byla realizována souběžně pro obě prostředí – prostřednictvím rozhraní Health Connect na Androidu a Apple HealthKit na iOS. Serverová část, jejíž existence ze zadání přímo nevyplývala, byla vyvinuta jako plnohodnotný systém zahrnující REST API s 52 explicitně definovanými endpointy, veřejnou webovou vrstvu, dokumentaci ve standardu OpenAPI a průběžné nasazení prostřednictvím GitHub Actions.

Aplikace je provozována na produkční cloudové infrastruktuře Oracle Cloud na adrese `lifemeter.fit` a je dostupná ke stažení prostřednictvím webového download centra ve formě verzovaných APK buildů. Stejná webová vrstva současně poskytuje veřejnou prezentaci produktu a administrátorskou konzoli pouze pro čtení. Uživateli nabízí jednotné prostředí pro oblasti, které stávající řešení na trhu pokrývají odděleně a zpravidla za poplatek – evidenci silových tréninků s výkonnostními grafy, sledování výživy s databází přes milion potravin ověřených zdrojem USDA FoodData Central, záznamy spánku s importem fází z chytrých zařízení a přehledovou domovskou obrazovku kombinující všechny sledované metriky. Offline režim zajišťuje plnou funkčnost i bez aktivního síťového připojení.

Projekt přinesl praktickou zkušenost s celým vývojovým cyklem full-stack mobilního a webového systému – od návrhu datového modelu a architektonických rozhodnutí přes implementaci a automatizované nasazení až po produkční provoz. Zvolená architektura s důsledným oddělením klientské, serverové a datové vrstvy vytváří základ, který umožňuje aplikaci dále rozvíjet v směrech nastíněných v kapitole Diskuze.

Literatura

[1] M. Swan. „The Quantified Self: Fundamental Disruption in Big Data Science and Biological Discovery“. In: *Big Data* 1.2 (čvn. 2013). PMID: 27442063, s. 85–99. ISSN: 2167-6461. DOI: 10.1089/big.2012.0002. URL: https://journals.sagepub.com/doi/10.1089/big.2012.0002 (cit. 11. 03. 2026).

[2] *React Native Documentation*. 2026. URL: https://reactnative.dev/ (cit. 12. 03. 2026).

[3] 650 Industries, Inc. *Expo Documentation*. 2026. URL: https://docs.expo.dev/ (cit. 12. 03. 2026).

[4] Jarred Sumner. *Bun Documentation*. 2026. URL: https://bun.sh/docs (cit. 12. 03. 2026).

[5] Google LLC. *Health Connect – Android Developers Documentation*. 2024. URL: https://developer.android.com/health-and-fitness/guides/health-connect (cit. 15. 03. 2026).

[6] Apple Inc. *HealthKit – Apple Developer Documentation*. 2024. URL: https://developer.apple.com/documentation/healthkit (cit. 15. 03. 2026).

[7] Gregor Hoff a Martin Kleppmann. *Offline First – Designing for Unreliable Networks*. 2015. URL: https://offlinefirst.org/ (cit. 15. 03. 2026).

[8] Tencent. *MMKV – Fast, Persistent Key-Value Storage*. 2024. URL: https://github.com/Tencent/MMKV (cit. 15. 03. 2026).

[9] Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm (cit. 15. 03. 2026).

[10] OWASP Foundation. *OWASP Mobile Application Security Verification Standard*. 2024. URL: https://mas.owasp.org/MASVS/ (cit. 15. 03. 2026).

[11] Google LLC. *Google Fit: Activity Tracking*. 2024. URL: https://www.google.com/fit/ (cit. 15. 03. 2026).

[12] Apple Inc. *Apple Health*. 2024. URL: https://www.apple.com/health/ (cit. 15. 03. 2026).

[13] MyFitnessPal, Inc. *MyFitnessPal: Calorie Counter and Diet Tracker*. 2024. URL: https://www.myfitnesspal.com/ (cit. 15. 03. 2026).

[14] Cronometer Software Inc. *Cronometer: Track Nutrition & Count Calories*. 2024. URL: https://cronometer.com/ (cit. 15. 03. 2026).

[15] Sleep Cycle AB. *Sleep Cycle: Sleep Tracker*. 2024. URL: https://www.sleepcycle.com/ (cit. 15. 03. 2026).

[16] Yusuke Wada. *Hono Documentation*. 2024. URL: https://hono.dev/docs/ (cit. 15. 03. 2026).

[17] Paul Bratslavsky. *Bun vs Node.js: A Practical Comparison Every Developer Should Read*. 2025. URL: https://strapi.io/blog/bun-vs-nodejs-performance-comparison-guide (cit. 16. 03. 2026).

[18] The PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*. 2024. URL: https://www.postgresql.org/docs/17/ (cit. 15. 03. 2026).

[19] Better Auth Contributors. *Better Auth Documentation*. 2024. URL: https://www.better-auth.com/docs (cit. 15. 03. 2026).

[20] Docker, Inc. *Docker Compose Documentation*. 2024. URL: https://docs.docker.com/compose/ (cit. 15. 03. 2026).

[21] The PostgreSQL Global Development Group. *pg_trgm – PostgreSQL Extension for Trigram Matching*. 2024. URL: https://www.postgresql.org/docs/17/pgtrgm.html (cit. 15. 03. 2026).

[22] U.S. Department of Agriculture. *FoodData Central*. 2024. URL: https://fdc.nal.usda.gov/ (cit. 15. 03. 2026).

[23] Mark D. Mifflin et al. „A new predictive equation for resting energy expenditure in healthy individuals“. In: *The American Journal of Clinical Nutrition* 51.2 (1990), s. 241–247. DOI: 10.1093/ajcn/51.2.241.

[24] Kevin D. Hall. „What is the required energy deficit per unit weight loss?“ In: *International Journal of Obesity* 32.3 (2008), s. 573–576. DOI: 10.1038/sj.ijo.0803720.

[25] Wilbur O. Atwater. *Principles of Nutrition and Nutritive Value of Food*. United States Department of Agriculture, 1902. URL: https://archive.org/details/principlesofnutr00atwa (cit. 18. 03. 2026).

Seznam obrázků

2.1 Přehled architektury systému LifeMeter 14
2.2 Entity-relationship diagram databázového schématu 17
2.3 Diagram případů užití aplikace LifeMeter 23
2.4 Diagram navigační struktury aplikace LifeMeter 24
2.5 Diagram tříd – doménové Context Providers klientské vrstvy 25
2.6 Uvítací obrazovka onboardingu 34
2.7 Domovská obrazovka – 7denní přehled 34
2.8 Modul výživy – denní makra a přehled jídel 34
2.9 Vyhledávání potravin a skenování čárového kódu 34
2.10 Modul spánku – přehled a záznam 35
2.11 Modul tréninku – aktivní trénink se sadami 35
2.12 Sestavení tréninkové šablony 35
2.13 Nastavení – notifikace a zdravotní integrace 35

Seznam tabulek

2.1 Přehled plnění požadavků zadání 31
2.2 Přehled technických parametrů aplikace 33
2.3 Tabulka `account` 48
2.4 Tabulka `activity_level` 48
2.5 Tabulka `branded_food` 49
2.6 Tabulka `exercise` 49
2.7 Tabulka `exercise_type` 49
2.8 Tabulka `exercise_variant` 49
2.9 Tabulka `food` 49
2.10 Tabulka `food_category` 49
2.11 Tabulka `length_unit` 50
2.12 Tabulka `muscle` 50
2.13 Tabulka `muscle_involvement` 50
2.14 Tabulka `nutrient` 50
2.15 Tabulka `nutrient_value` 50
2.16 Tabulka `portion` 50
2.17 Tabulka `raw_muscles` 51
2.18 Tabulka `session` 51
2.19 Tabulka `set_style` 51
2.20 Tabulka `set_type` 51
2.21 Tabulka `template_workout_set` 52
2.22 Tabulka `user` 52
2.23 Tabulka `user_blood_pressure_log` 53
2.24 Tabulka `user_food` 53
2.25 Tabulka `user_goal` 53
2.26 Tabulka `user_health_sync_pending_run` 54
2.27 Tabulka `user_health_sync_source_item` 55
2.28 Tabulka `user_health_sync_state` 56
2.29 Tabulka `user_heart_rate_log` 56
2.30 Tabulka `user_height_log` 56

2.31 Tabulka `user_meal` 56
2.32 Tabulka `user_profile` 57
2.33 Tabulka `user_sleep` 57
2.34 Tabulka `user_weight_log` 57
2.35 Tabulka `verification` 58
2.36 Tabulka `weight_unit` 58
2.37 Tabulka `workout` 58
2.38 Tabulka `workout_set` 59
2.39 Tabulka `workout_template` 59

Seznam příloh

A Diagramy

2.16 Tabulky

Sloupec	Datový typ	Popis
id	TEXT	Unikátní identifikátor záznamu
accountId	TEXT	Identifikátor externího účtu
providerId	TEXT	Identifikátor poskytovatele autentizace
userId	TEXT	Unikátní identifikátor uživatele
accessToken	TEXT	Přístupový token pro autentizaci
refreshToken	TEXT	Obnovovací token pro autentizaci
idToken	TEXT	Token identity
accessTokenExpiresAt	TIMESTAMP WITH TIME ZONE	Datum a čas expirace přístupového tokenu
refreshTokenExpiresAt	TIMESTAMP WITH TIME ZONE	Datum a čas expirace obnovovacího tokenu
scope	TEXT	Rozsah oprávnění tokenu
password	TEXT	Hash hesla uživatele
createdAt	TIMESTAMP WITH TIME ZONE	Datum a čas vytvoření záznamu
updatedAt	TIMESTAMP WITH TIME ZONE	Datum a čas poslední aktualizace záznamu

Tabulka 2.3: Tabulka `account`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
name	TEXT	Název úrovně aktivity
description	TEXT	Podrobný popis úrovně aktivity
min_factor	NUMERIC	Minimální koeficient pro výpočet energetického výdeje
max_factor	NUMERIC	Maximální koeficient pro výpočet energetického výdeje

Tabulka 2.4: Tabulka `activity_level`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
brand_owner	CHARACTER VARYING(128)	Vlastník obchodní značky
brand_name	CHARACTER VARYING(128)	Název obchodní značky
subbrand_name	CHARACTER VARYING(128)	Název podznačky
gtin_upc	CHARACTER VARYING(32)	Čárový kód produktu (GTIN/UPC)
ingredients	TEXT	Seznam ingrediencí a složení

Tabulka 2.5: Tabulka `branded_food`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
exercise_type_id	UUID	Identifikátor typu cviku
exercise_variant_id	UUID	Identifikátor varianty cviku

Tabulka 2.6: Tabulka `exercise`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
name	TEXT	Název typu cviku

Tabulka 2.7: Tabulka `exercise_type`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
name	TEXT	Název varianty cviku

Tabulka 2.8: Tabulka `exercise_variant`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
branded_food_id	INTEGER	Identifikátor značkové potraviny (volitelné)
food_category_id	INTEGER	Identifikátor kategorie potraviny
description	TEXT	Podrobný název nebo popis potraviny
search_vector	TSVECTOR	Vektor pro plnotextové vyhledávání

Tabulka 2.9: Tabulka `food`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
name	TEXT	Název kategorie potravin

Tabulka 2.10: Tabulka `food_category`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
name	TEXT	Název jednotky délky
meter_conversion_factor	NUMERIC	Koeficient pro převod na metry

Tabulka 2.11: Tabulka `length_unit`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
name	TEXT	Název svalu
action	TEXT	Funkce nebo pohyb svalu
parent_id	UUID	Identifikátor nadřazené svalové skupiny

Tabulka 2.12: Tabulka `muscle`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
exercise_id	UUID	Identifikátor cviku
muscle_id	UUID	Identifikátor svalu
involvement	NUMERIC	Míra zapojení svalu při daném cviku

Tabulka 2.13: Tabulka `muscle_involvement`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
name	CHARACTER VARYING(128)	Název živiny
unit	USER-DEFINED	Měrná jednotka živiny
nutrient_nbr	INTEGER	Číselné označení živiny

Tabulka 2.14: Tabulka `nutrient`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
food_id	INTEGER	Identifikátor potraviny
nutrient_id	INTEGER	Identifikátor živiny
amount	NUMERIC	Množství živiny v potravině

Tabulka 2.15: Tabulka `nutrient_value`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
food_id	INTEGER	Identifikátor potraviny
gram_weight	NUMERIC	Hmotnost porce v gramech
portion_amount	NUMERIC	Množství jednotek v porci
portion_unit	CHARACTER VARYING(16)	Označení jednotky porce
description	TEXT	Popis porce
modifier	TEXT	Upřesňující informace k porci

Tabulka 2.16: Tabulka `portion`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
muscle	TEXT	Název svalu
location	TEXT	Anatomické umístění
origin	TEXT	Anatomický začátek svalu
insertion	TEXT	Anatomický úpon svalu
artery	TEXT	Zásobující tepna
nerve	TEXT	Inervující nerv
action	TEXT	Funkce svalu
antagonist	TEXT	Protichůdně působící sval
o	INTEGER	Parametr o
t	INTEGER	Parametr t

Tabulka 2.17: Tabulka `raw_muscles`

Sloupec	Datový typ	Popis
id	TEXT	Unikátní identifikátor relace
expiresAt	TIMESTAMP WITH TIME ZONE	Datum a čas expirace relace
token	TEXT	Autentizační token relace
createdAt	TIMESTAMP WITH TIME ZONE	Datum a čas vytvoření relace
updatedAt	TIMESTAMP WITH TIME ZONE	Datum a čas poslední aktualizace relace
ipAddress	TEXT	IP adresa uživatele
userAgent	TEXT	Identifikace prohlížeče nebo zařízení
userId	TEXT	Unikátní identifikátor uživatele
impersonatedBy	TEXT	Identifikátor uživatele provádějícího impersonaci

Tabulka 2.18: Tabulka `session`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
name	TEXT	Název stylu série (např. běžná, drop set)

Tabulka 2.19: Tabulka `set_style`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
name	TEXT	Název typu série (např. zahřívací, pracovní)

Tabulka 2.20: Tabulka `set_type`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
workout_template_id	UUID	Identifikátor šablony tréninku
exercise_id	UUID	Identifikátor cviku
seq_number	INTEGER	Pořadové číslo série v tréninku
repetitions	INTEGER	Počet opakování v sérii
rir	INTEGER	Rezerva v opakování (Reps In Reserve)
rest_time	INTERVAL	Doba odpočinku po sérii
notes	TEXT	Poznámky k sérii
style_id	UUID	Identifikátor stylu série
set_type_id	UUID	Identifikátor typu série

Tabulka 2.21: Tabulka `template_workout_set`

Sloupec	Datový typ	Popis
id	TEXT	Unikátní identifikátor záznamu
name	TEXT	Celé jméno uživatele
email	TEXT	E-mailová adresa uživatele
emailVerified	BOOLEAN	Příznak, zda byl e-mail uživatele ověřen
image	TEXT	Odkaz na profilový obrázek uživatele
createdAt	TIMESTAMP WITH TIME ZONE	Datum a čas registrace uživatele
updatedAt	TIMESTAMP WITH TIME ZONE	Datum a čas poslední aktualizace profilu
role	TEXT	Role uživatele v systému
banned	BOOLEAN	Příznak, zda je uživatel zablokován
banReason	TEXT	Důvod zablokování uživatele
banExpires	TIMESTAMP WITH TIME ZONE	Datum a čas vypršení zablokování
lastLoginMethod	TEXT	Poslední použitý způsob přihlášení
normalizedEmail	TEXT	Normalizovaná verze e-mailové adresy

Tabulka 2.22: Tabulka `user`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_id	TEXT	Unikátní identifikátor uživatele
measured_at	TIMESTAMP WITH TIME ZONE	Datum a čas měření krevního tlaku
systolic_mmhg	INTEGER	Systolický krevní tlak (mmHg)
diastolic_mmhg	INTEGER	Diastolický krevní tlak (mmHg)

Tabulka 2.23: Tabulka `user_blood_pressure_log`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_meal_id	UUID	Identifikátor jídla uživatele
food_id	INTEGER	Identifikátor potraviny z databáze
total_grams	NUMERIC	Celková hmotnost zkonzumované potraviny v gramech
quantity	NUMERIC	Množství porcí
portion_id	INTEGER	Identifikátor použité porce
description	TEXT	Vlastní popis nebo poznámka k potravině

Tabulka 2.24: Tabulka `user_food`

Sloupec	Datový typ	Popis
user_id	TEXT	Unikátní identifikátor uživatele
daily_steps_goal	INTEGER	Cílový denní počet kroků
bedtime_goal	TIME	Cílový čas odchodu ke spánku
wakeup_goal	TIME	Cílový čas probuzení
daily_protein_goal_grams	INTEGER	Cílový denní příjem bílkovin v gramech
daily_fat_goal_grams	INTEGER	Cílový denní příjem tuků v gramech
daily_carbs_goal_grams	INTEGER	Cílový denní příjem sacharidů v gramech
target_weight_grams	NUMERIC	Cílová tělesná hmotnost v gramech
target_weight_date	DATE	Cílové datum pro dosažení hmotnosti

Tabulka 2.25: Tabulka `user_goal`

Sloupec	**Datový typ**	**Popis**
user_id	TEXT	Unikátní identifikátor uživatele
provider	TEXT	Identifikátor zdravotního providera synchronizace
sync_run_id	TEXT	Identifikátor rozpracovaného synchronizačního běhu
expected_batch_index	INTEGER	Index další očekávané dávky při pokračování běhu
current_sync_state	JSONB	Uložený stav synchronizace před spuštěním dávkového běhu
next_sync_state	JSONB	Stav synchronizace připravený ke commitnutí po dokončení běhu
created_at	TIMESTAMP WITH TIME ZONE	Datum a čas vytvoření záznamu
updated_at	TIMESTAMP WITH TIME ZONE	Datum a čas poslední aktualizace záznamu

Tabulka 2.26: Tabulka `user_health_sync_pending_run`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor mapovaného zdrojového záznamu
user_id	TEXT	Unikátní identifikátor uživatele
provider	TEXT	Identifikátor zdravotního providera synchronizace
source_type	TEXT	Typ synchronizované entity z externí platformy
source_item_id	TEXT	Identifikátor zdrojového záznamu u providera
target_table	TEXT	Název cílové aplikační tabulky
target_row_id	UUID	Identifikátor cílového řádku v aplikační tabulce
raw_payload	JSONB	Původní payload převzatý od providera
checksum	TEXT	Kontrolní součet normalizovaného zdrojového záznamu
source_start_at	TIMESTAMP WITH TIME ZONE	Začátek intervalu zdrojového záznamu
source_end_at	TIMESTAMP WITH TIME ZONE	Konec intervalu zdrojového záznamu
source_last_modified_at	TIMESTAMP WITH TIME ZONE	Čas poslední změny zdrojového záznamu u providera
owns_target	BOOLEAN	Příznak, zda je cílový řádek vlastněn právě tímto zdrojem
created_at	TIMESTAMP WITH TIME ZONE	Datum a čas vytvoření záznamu
updated_at	TIMESTAMP WITH TIME ZONE	Datum a čas poslední aktualizace záznamu

Tabulka 2.27: Tabulka `user_health_sync_source_item`

Sloupec	Datový typ	Popis
user_id	TEXT	Unikátní identifikátor uživatele
provider	TEXT	Identifikátor zdravotního providera synchronizace
sync_state	JSONB	Serializovaný stav poslední úspěšné synchronizace
last_success_at	TIMESTAMP WITH TIME ZONE	Datum a čas posledního úspěšného běhu synchronizace
created_at	TIMESTAMP WITH TIME ZONE	Datum a čas vytvoření záznamu
updated_at	TIMESTAMP WITH TIME ZONE	Datum a čas poslední aktualizace záznamu
owns_target	BOOLEAN	Indikátor, zda uživatel vlastní cílový účet u providera

Tabulka 2.28: Tabulka `user_health_sync_state`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_id	TEXT	Unikátní identifikátor uživatele
measured_at	TIMESTAMP WITH TIME ZONE	Datum a čas měření tepové frekvence
bpm	INTEGER	Počet tepů za minutu (Beats Per Minute)

Tabulka 2.29: Tabulka `user_heart_rate_log`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_id	TEXT	Unikátní identifikátor uživatele
measured_at	TIMESTAMP WITH TIME ZONE	Datum a čas měření výšky
height_cm	NUMERIC	Tělesná výška v centimetrech

Tabulka 2.30: Tabulka `user_height_log`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_id	TEXT	Unikátní identifikátor uživatele
eaten_at	TIMESTAMP WITH TIME ZONE	Datum a čas konzumace jídla
name	TEXT	Název jídla (např. Snídaně, Oběd)

Tabulka 2.31: Tabulka `user_meal`

Sloupec	Datový typ	Popis
user_id	TEXT	Unikátní identifikátor uživatele
date_of_birth	DATE	Datum narození uživatele
sex	CHARACTER(1)	Pohlaví uživatele (M/F)
current_activity_factor	NUMERIC	Aktuálně nastavený koeficient aktivity
current_bmr_calories	INTEGER	Aktuální hodnota bazálního metabolismu v kaloriích
default_weight_unit_id	INTEGER	Preferovaná jednotka hmotnosti
default_length_unit_id	INTEGER	Preferovaná jednotka délky
finished_onboarding	BOOLEAN	Příznak dokončení úvodního nastavení (onboarding)

Tabulka 2.32: Tabulka `user_profile`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_id	TEXT	Unikátní identifikátor uživatele
sleep_start	TIMESTAMP WITH TIME ZONE	Datum a čas začátku spánku
sleep_end	TIMESTAMP WITH TIME ZONE	Datum a čas konce spánku
note	TEXT	Poznámka ke kvalitě nebo průběhu spánku

Tabulka 2.33: Tabulka `user_sleep`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_id	TEXT	Unikátní identifikátor uživatele
measured_at	TIMESTAMP WITH TIME ZONE	Datum a čas měření hmotnosti
weight_grams	NUMERIC	Tělesná hmotnost v gramech
body_fat_percentage	NUMERIC	Procento tělesného tuku
lean_tissue_percentage	NUMERIC	Procento beztukové tkáně
water_percentage	NUMERIC	Procento vody v těle
bone_mass_percentage	NUMERIC	Procento kostní hmoty

Tabulka 2.34: Tabulka `user_weight_log`

Sloupec	Datový typ	Popis
id	TEXT	Unikátní identifikátor záznamu
identifier	TEXT	Identifikátor pro ověření (např. e-mail)
value	TEXT	Hodnota ověřovacího kódu nebo tokenu
expiresAt	TIMESTAMP WITH TIME ZONE	Datum a čas expirace ověření
createdAt	TIMESTAMP WITH TIME ZONE	Datum a čas vytvoření požadavku na ověření
updatedAt	TIMESTAMP WITH TIME ZONE	Datum a čas poslední aktualizace požadavku

Tabulka 2.35: Tabulka `verification`

Sloupec	Datový typ	Popis
id	INTEGER	Unikátní identifikátor záznamu
name	TEXT	Název jednotky hmotnosti
gram_conversion_factor	NUMERIC	Koeficient pro převod na gramy

Tabulka 2.36: Tabulka `weight_unit`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_id	TEXT	Unikátní identifikátor uživatele
workout_template_id	UUID	Identifikátor použité šablony tréninku
start_date	TIMESTAMP WITH TIME ZONE	Datum a čas zahájení tréninku
end_date	TIMESTAMP WITH TIME ZONE	Datum a čas ukončení tréninku
label	ARRAY	Štítky pro kategorizaci tréninku
notes	TEXT	Poznámky k průběhu tréninku

Tabulka 2.37: Tabulka `workout`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
workout_id	UUID	Identifikátor tréninku
exercise_id	UUID	Identifikátor cviku
seq_number	INTEGER	Pořadové číslo série v tréninku
weight	NUMERIC	Hmotnost použitá v sérii
weight_unit_id	INTEGER	Identifikátor jednotky hmotnosti
repetitions	INTEGER	Počet provedených opakování
rir	INTEGER	Rezerva v opakování (Reps In Reserve)
rest_time	INTERVAL	Doba odpočinku po sérii
notes	TEXT	Poznámky k sérii
style_id	UUID	Identifikátor stylu série
set_type_id	UUID	Identifikátor typu série

Tabulka 2.38: Tabulka `workout_set`

Sloupec	Datový typ	Popis
id	UUID	Unikátní identifikátor záznamu
user_id	TEXT	Unikátní identifikátor uživatele
name	TEXT	Název šablony tréninku
description	TEXT	Podrobný popis šablony
label	ARRAY	Štítky pro kategorizaci šablony
created_at	TIMESTAMP WITH TIME ZONE	Datum a čas vytvoření šablony
updated_at	TIMESTAMP WITH TIME ZONE	Datum a čas poslední aktualizace šablony

Tabulka 2.39: Tabulka `workout_template`

C Schémata

D API dokumentace

E Instalační návod